误差逆传播算法

误差逆传播算法（Error BackPropagation，简称BP）是最成功的训练多层前馈神经网络的学习算法.

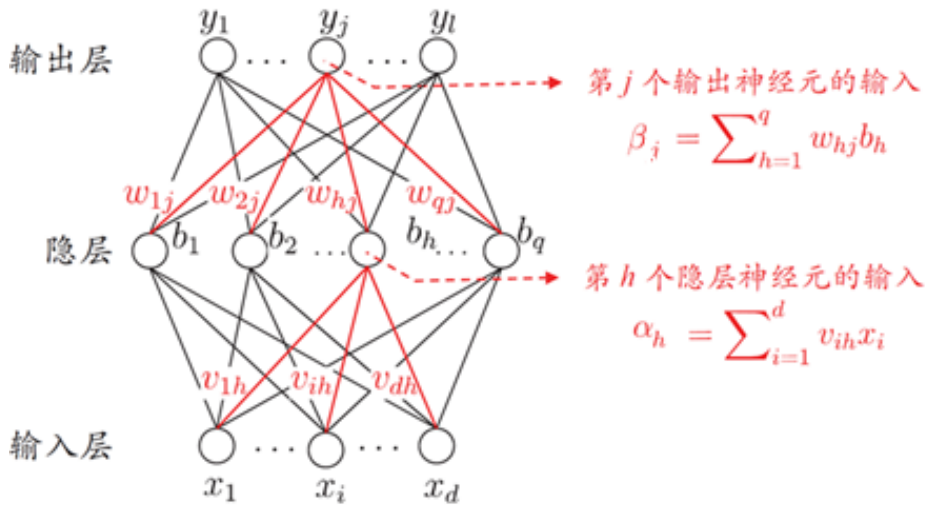
- 给定训练集$D=\{(\boldsymbol{x}_i,y_i)\},\boldsymbol{x}_i\in R^d,y_i\in R^l,(i=1,2,\cdots,m)$，即输入实例由$d$个属性描述，输出$l$维实值向量.
- 给定一个拥有d个输入神经元，l个输出神经元，q个隐层神经元的多层前向前馈网络结构.
- 记号：

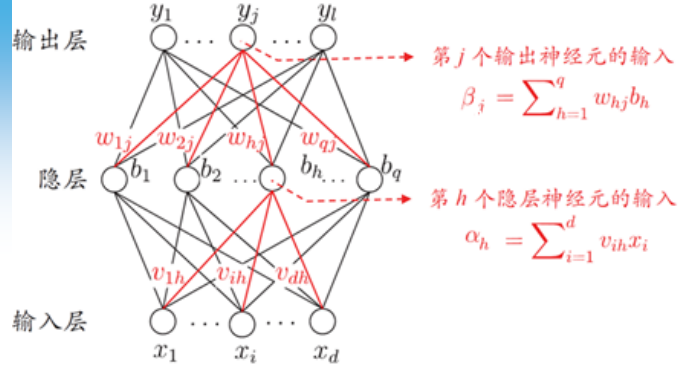
θ_j：输出层第j个神经元阈值；

γ_h：隐含层第h个神经元阈值；

v_{ih}：输入层与隐层神经元之间连接权重；

w_{hj}：隐层与输出层神经元之间连接权重；





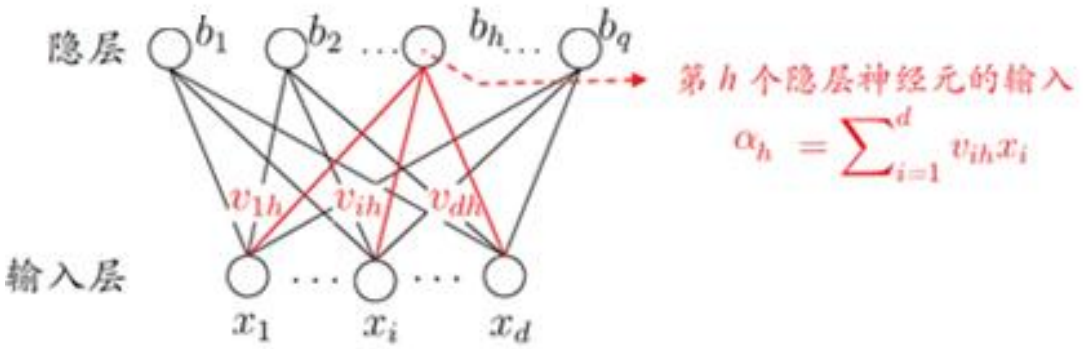
对于样例$(\boldsymbol{x}_k, \boldsymbol{y}_k)$,

假设网络的实际输出为$\hat{\boldsymbol{y}}_k$

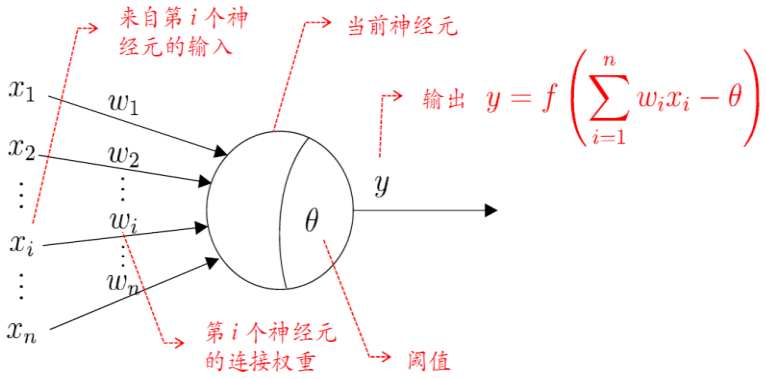
前向计算

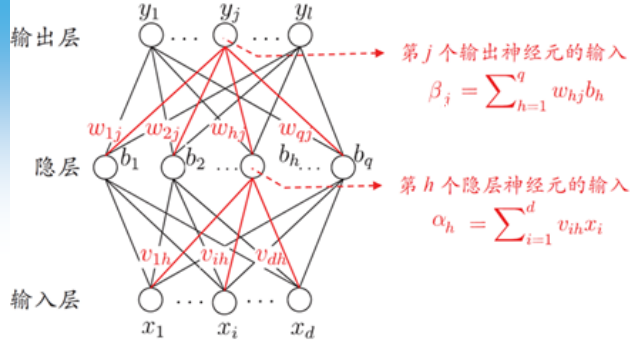
step1:

$$b_h = f(\alpha_h - \gamma_h)$$



$$\alpha_h = \sum_{i=1}^{d} v_{ih} x_i$$



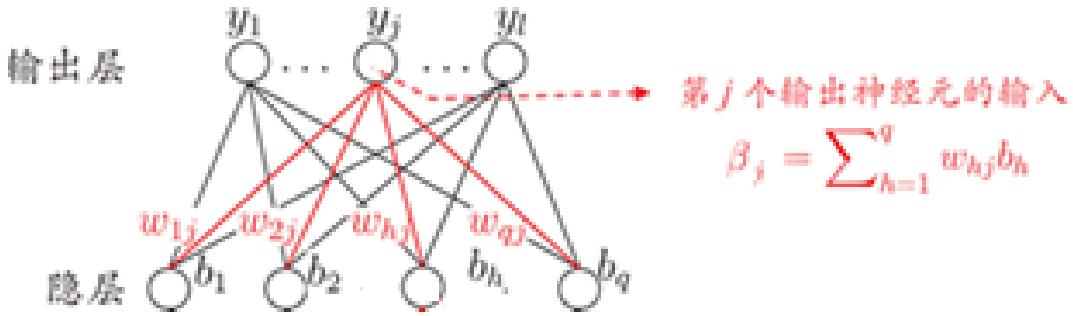


step2:

$$\hat{y}_j^k = f(\beta_j - \theta_j),\ \beta_j = \sum_{h=1}^{q} w_{hj} b_h$$

step3:

$$E_k = \frac{1}{2}\sum_{j=1}^{l}(\hat{y}_j^k - y_j^k)^2$$



参数数目

权重：v_{ih} ，w_{hj} 阈值：θ_j ，γ_h $(i = 1, \cdots, d, h = 1, , \cdots, q, j = 1, \cdots, l)$

因此网络中需要 $(d + l + 1)q + l$ 个参数需要优化

参数优化

BP是一个迭代学习算法，在迭代的每一轮中采用广义的感知机学习规则对参数进行更新估计，任意的参数 v 的更新估计式为 $v \leftarrow v + \triangle v.$

BP 学习算法

BP算法基于**梯度下降策略**，以目标的负梯度方向对参数进行调整．对误差 E_k，给定学习率 η

$$\triangle w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}}$$

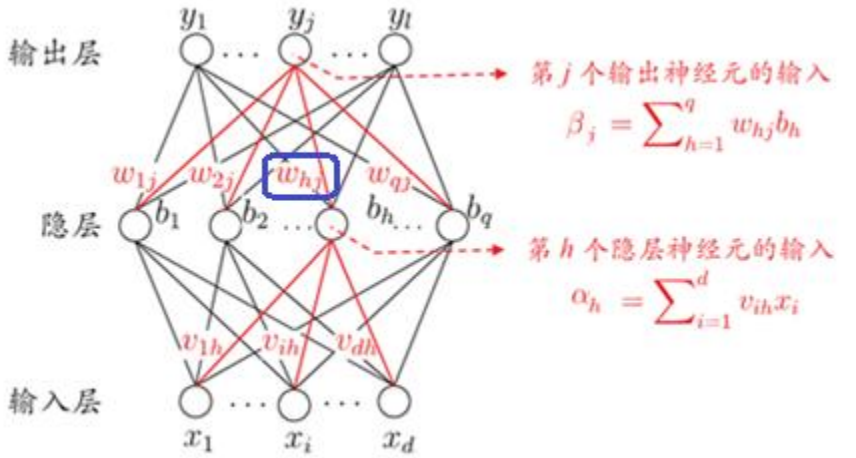
$$E_k = \frac{1}{2}\sum_{j=1}^{l}(\hat{y}_j^k - y_j^k)^2$$

$$\hat{y}_j^k = f(\beta_j - \theta_j),\ \beta_j = \sum_{h=1}^{q} w_{hj} b_h$$

$$\frac{\partial E_k}{\partial w_{hj}} = \boxed{\frac{\partial E_k}{\partial \hat{y}_j^k}} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \boxed{\frac{\partial \beta_j}{\partial w_{hj}}}$$

$$(\hat{y}_j^k - y_j^k) \qquad f'(\beta_j - \theta_j) \qquad b_h$$

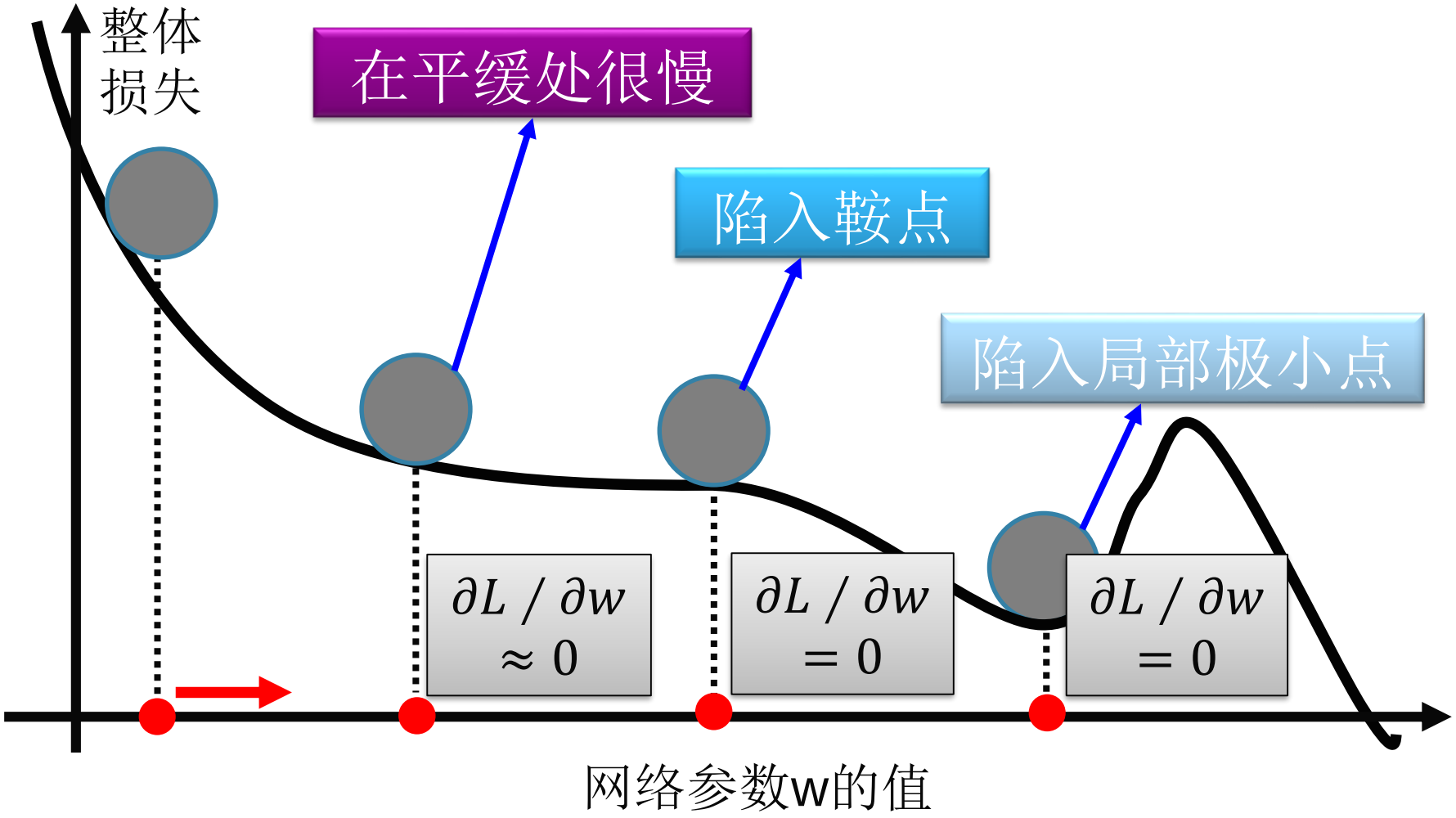
$$\hat{y}_j^k(1 - \hat{y}_j^k)$$



$$\begin{aligned} g_j &= -\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \\ &= -(\hat{y}_j^k - y_j^k) f'(\beta_j - \theta_j) \\ &= \hat{y}_j^k(1 - \hat{y}_j^k)(y_j^k - \hat{y}_j^k) \end{aligned}$$

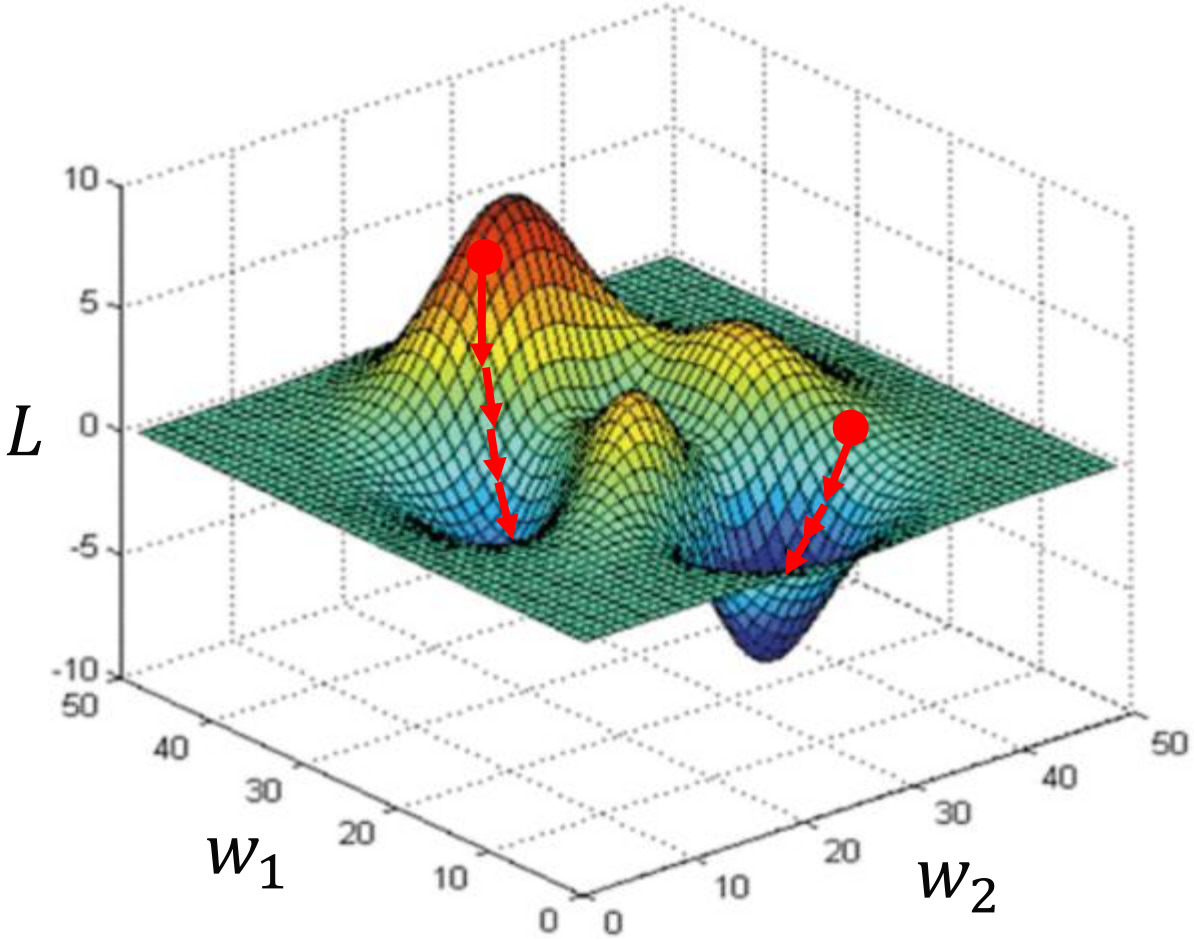
$$\triangle w_{hj} = \eta g_j b_h$$

局部极小点


整体
损失
在平缓处很慢
陷入鞍点
陷入局部极小点
$\partial L / \partial w \approx 0$
$\partial L / \partial w = 0$
$\partial L / \partial w = 0$
网络参数w的值

局部极小点

梯度下降法无法保证全局最小点



不同的起点

到达不同的极小点，即不同的结果

反向传播算法（Backpropagation）

一个神经网络有成百上千万个参数，反向传播算法是一种高效计算所有梯度$\partial L/\partial w$的算法。

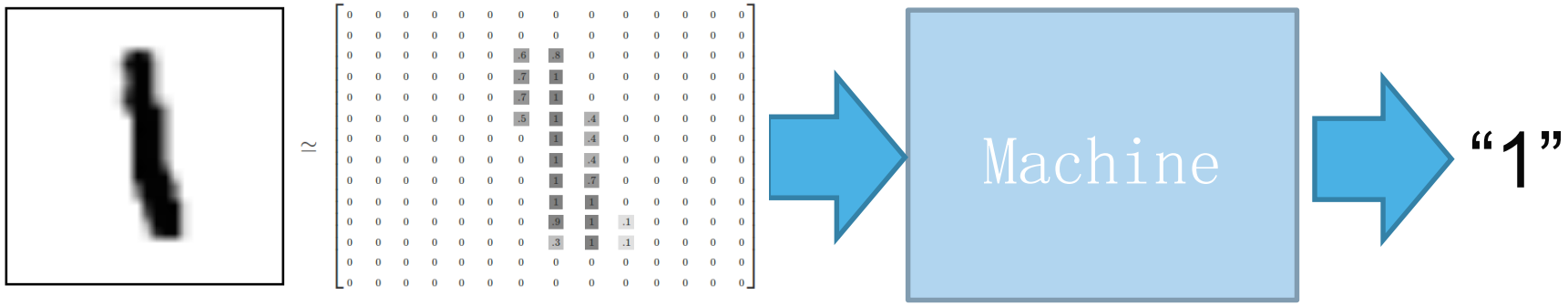
许多已有的深度学习框架可以自动计算梯度。

TensorFlow

PYTORCH

14.1 深度学习发展简史
14.2 多层感知机
14.3 损失函数
14.4 优化算法
14.5 案例：手写数字识别
14.6 深度学习技巧
14.7 卷积神经网络
14.8 案例：图像识别

案例应用：手写数字识别



28 x 28

MNIST 数据集: http://yann.lecun.com/exdb/mnist/

Keras

TensorFlow or PYTORCH

Very flexible

keras

Easy to learn and use

Keras：数据加载

Keras 提供了数据集加载函数： http://keras.io/datasets/

```
from keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

Keras：数据预处理

将每个图像矩阵扁平化成一个向量

```
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1]*X_train.shape[2])
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1]*X_test.shape[2])
```

将输入值从[0-255]归一化成[0-1]的值

```
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
X_train /= 255
X_test /= 255
```

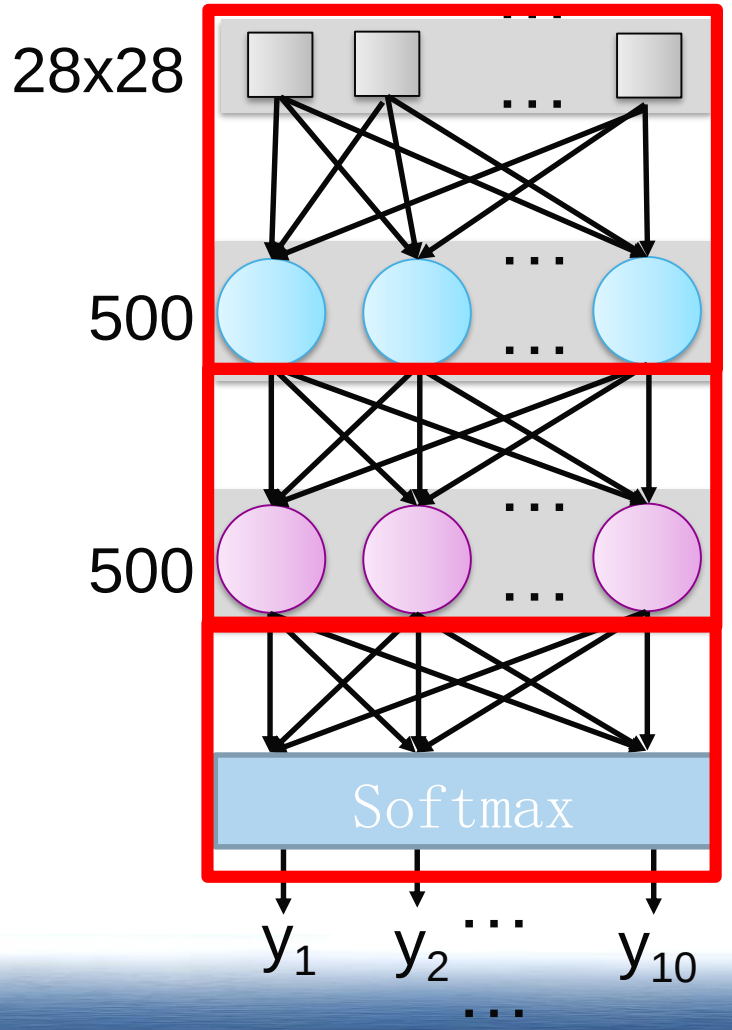
将每个类别值转换成一个10维的0-1向量

```
from keras.utils import np_utils

Y_train = np_utils.to_categorical(y_train, 10)
Y_test = np_utils.to_categorical(y_test, 10)
```

Keras

第一步：构建神经网络 → 第二步：确定学习目标 → 第三步：学习！




```
from keras.models import Sequential
from keras.layers import Dense,Activation

model = Sequential()
```

```
model.add(Dense(500, input_dim=784))
model.add(Activation('sigmoid'))
```

```
model.add(Dense(500))
model.add(Activation('sigmoid'))
```

```
model.add(Dense(10))
model.add(Activation('softmax'))
```

```
model.summary()
```

Keras

第一步：构建神经网络 → 第二步：确定学习目标 → 第三步：学习！

```
model.compile(loss = 'mean_squared_error',
              optimizer = 'sgd',
              metrics = ['accuracy'])
```

- loss：损失函数

  mean_squared_error、categorical_crossentropy…..
- optimizer：优化器

  sgd、rmsprop、adam……
- metrics：评价指标

Keras

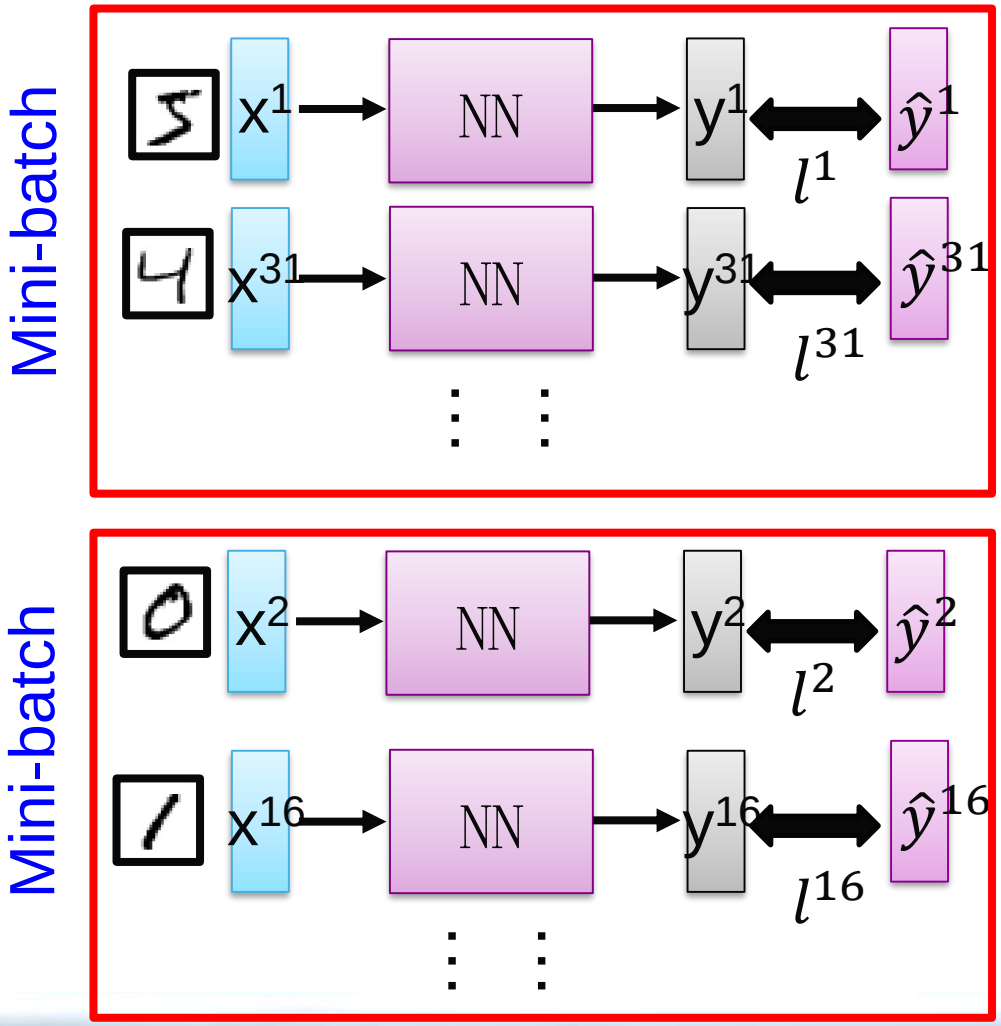
第一步：构建神经网络 → 第二步：确定学习目标 → 第三步：学习！

```
model.fit(X_train, Y_train, batch_size=100, epochs=20)
```

训练数据（图像）　类别标签（数字）　后面详细解释

小批量优化算法
Mini-batch

其实不是最小化整体损失！



- 随机初始化网络参数
- 用第一批数据
  $L' = l^1 + l^{31} + \cdots$
  修改一次参数
- 用第二批数据
  $L'' = l^2 + l^{16} + \cdots$
  修改一次参数
  ⋮
- 直到所有批的数据都使用过

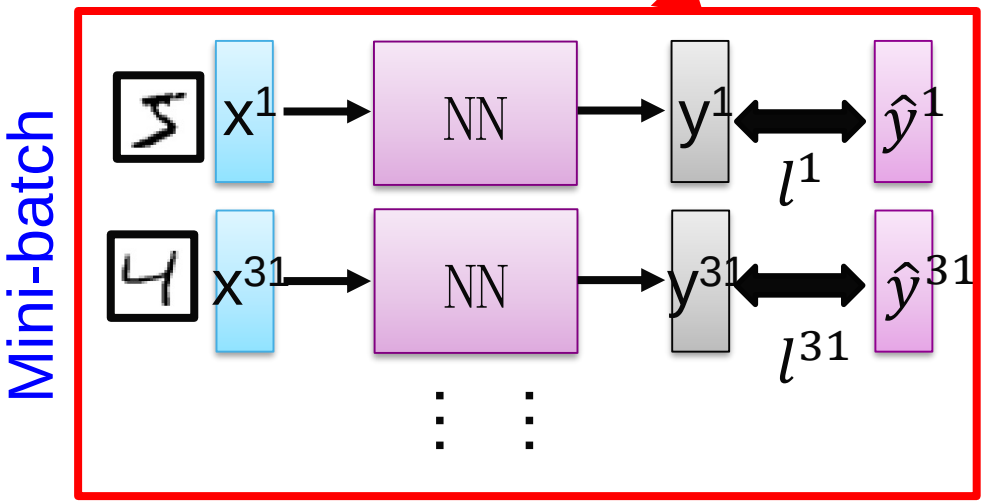
一个epoch

重复以上过程

小批量优化算法

Mini-batch

```
model.fit(X_train, Y_train, batch_size=100, epochs=20)
```



每批里有100个样例数据

- 用第一批数据
  $L' = l^1 + l^{31} + \cdots$
  修改一次参数
- 用第二批数据
  $L'' = l^2 + l^{16} + \cdots$
  修改一次参数

  ⋮
- 直到所有批的数据都使用过

重复20次

一个epoch

批量梯度下降（Batch Gradient Descent,BGD）

梯度下降的每一步中，都用到了所有的训练样本

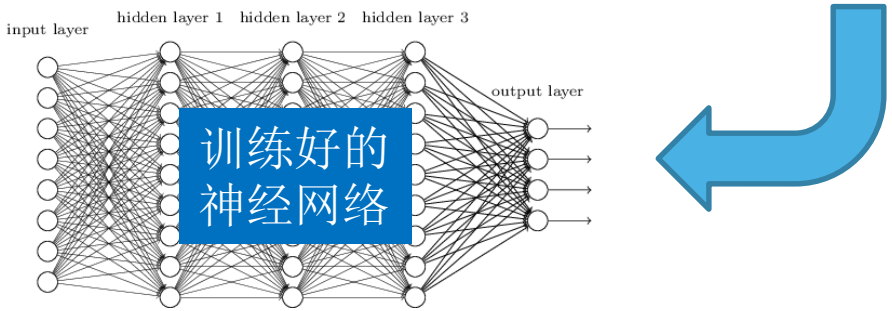
随机梯度下降（Stochastic Gradient Descent,SGD）

梯度下降的每一步中，用到一个样本，在每一次计算之后便更新参数，而不需要首先将所有的训练集求和

小批量梯度下降（Mini-Batch Gradient Descent,MBGD）

梯度下降的每一步中，用到了一定批量的训练样本

Keras



存储和加载训练好的模型：

http://keras.io/getting-started/faq/#how-can-i-save-a-keras-model

评估和使用训练好的模型：

评估:

```
score = model.evaluate(X_test, Y_test)
print('Test loss: ', score[0])
print('Test accuracy: ', score[1])
```

使用:

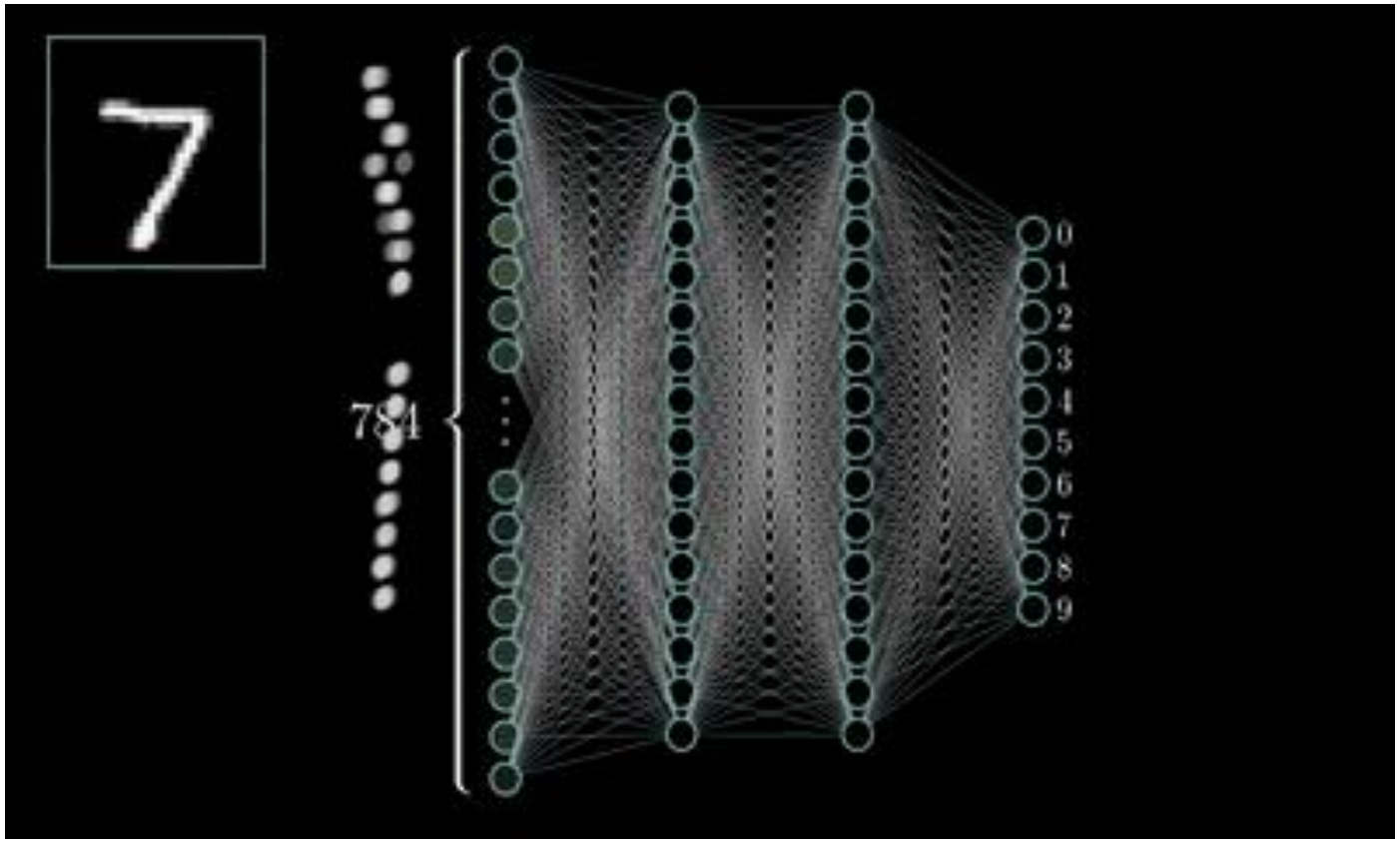
```
result = model.predict(X_test)
```

完整的程序

P166 手写数字识别

Test loss: 0.089

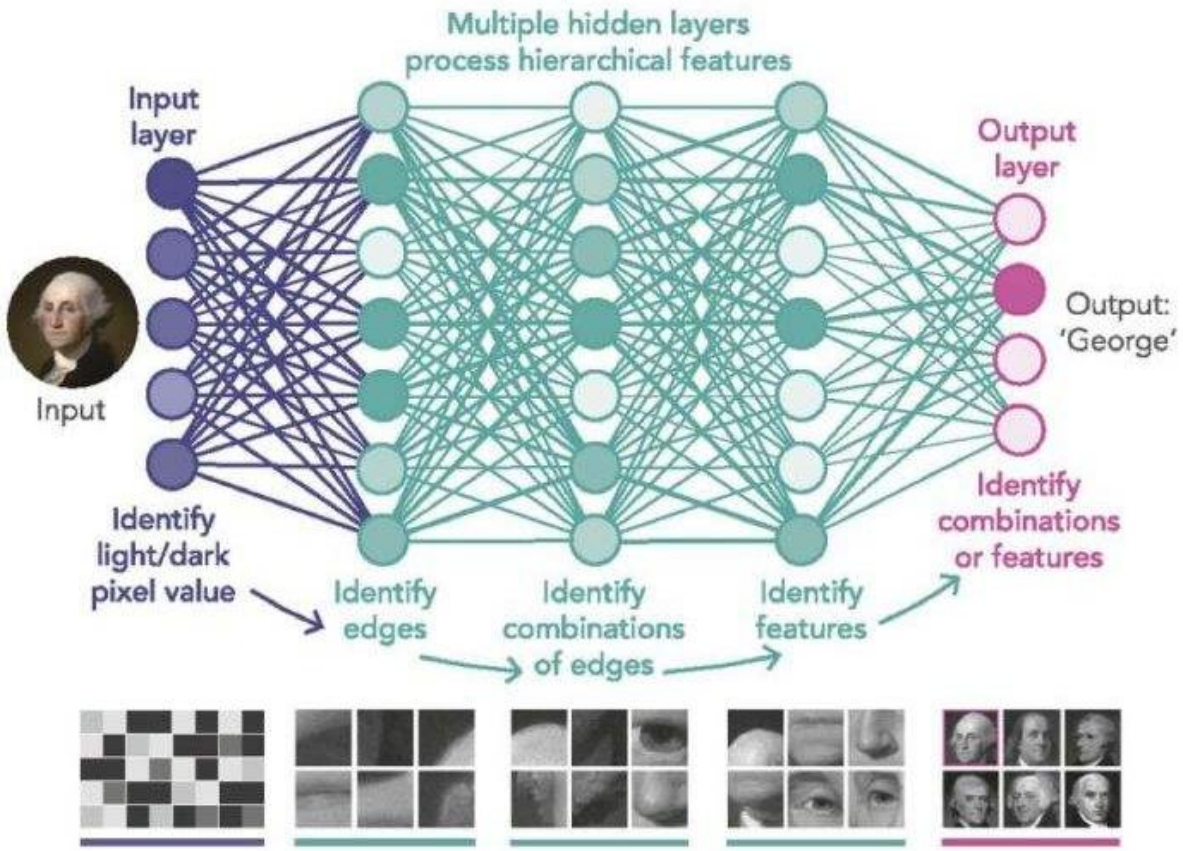
Test accuracy: 0.1425



14.1 深度学习发展简史
14.2 多层感知机
14.3 损失函数
14.4 优化算法
14.5 案例：手写数字识别
14.6 深度学习技巧
14.7 卷积神经网络
14.8 案例：图像识别

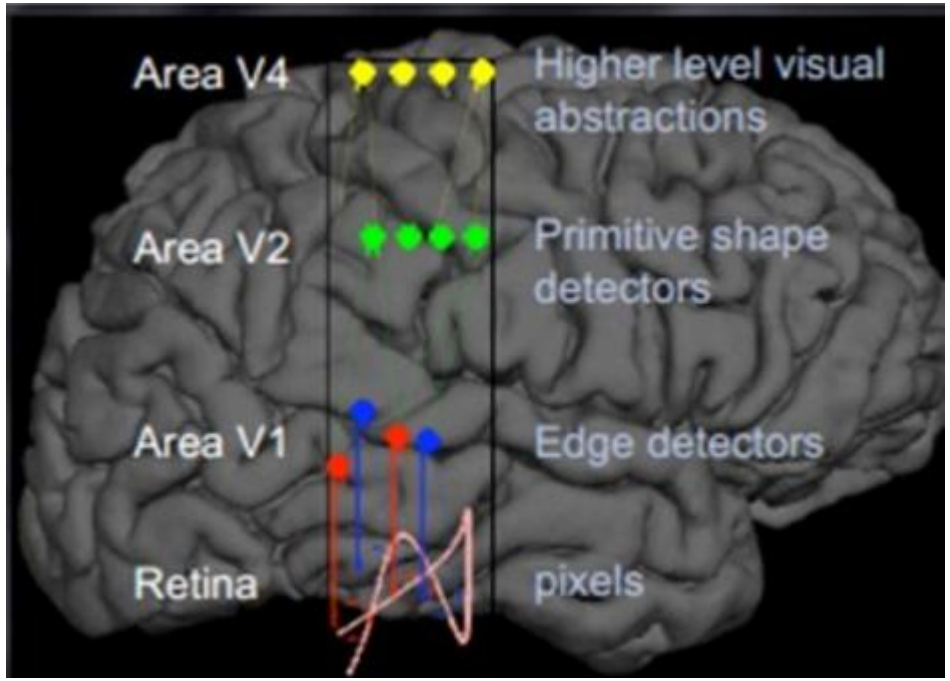
深度学习模型

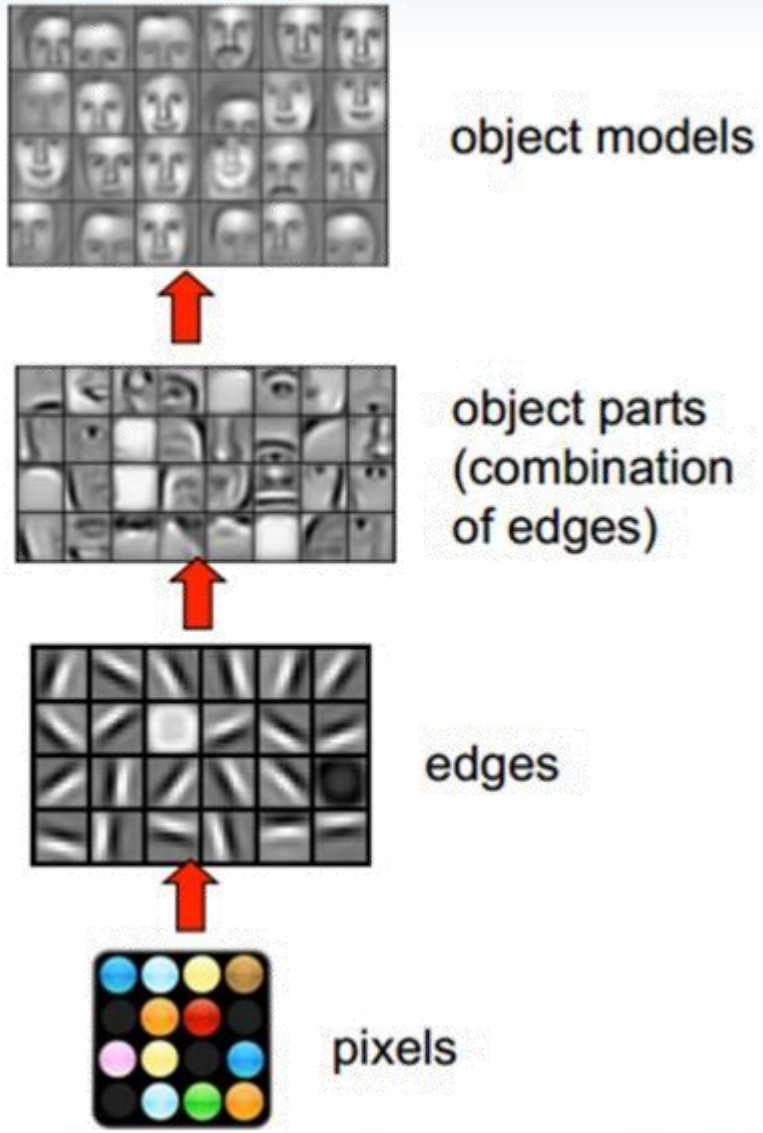
- 典型的深度学习模型就是很深层的神经网络.



人类视觉机理：

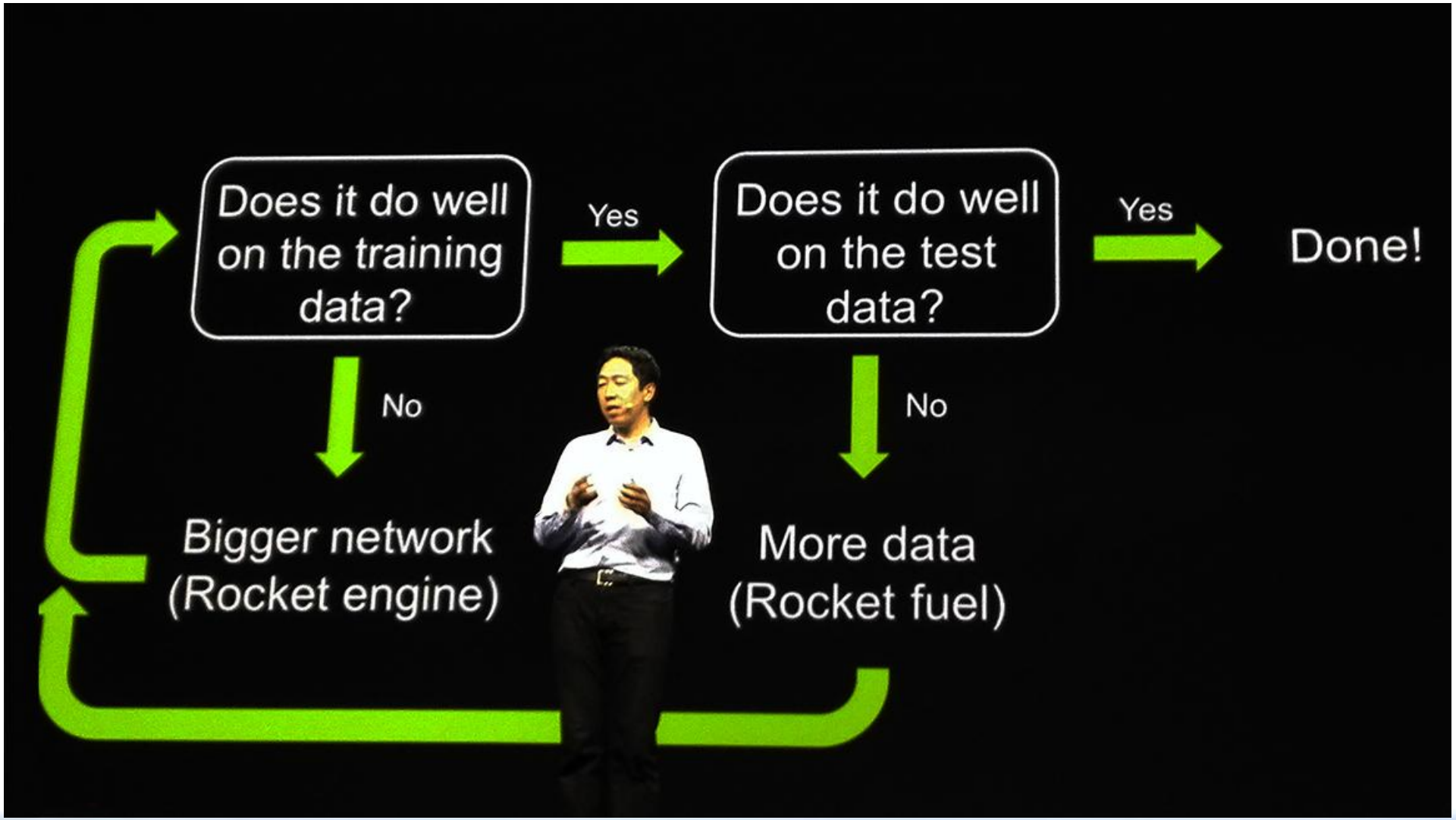
David Hubel和Torsten Wiesel发现了视觉系统的信息处理方式，即视皮层的分级特性，获得1981年诺贝尔生理学或医学奖。





Low-level sensing ⇨ **Preprocessing** ⇨ **Feature extraction** ⇨ **Feature selection** ⇨ **Inference: prediction, recognition**

深度学习技巧


Does it do well on the training data?
Yes
Does it do well on the test data?
Yes
Done!
No
No
Bigger network (Rocket engine)
More data (Rocket fuel)

深度学习技巧

YES

NO

Good Results on Testing Data?

- Dropout
- 正则化（Regularization）
- 提前终止（Early Stopping）

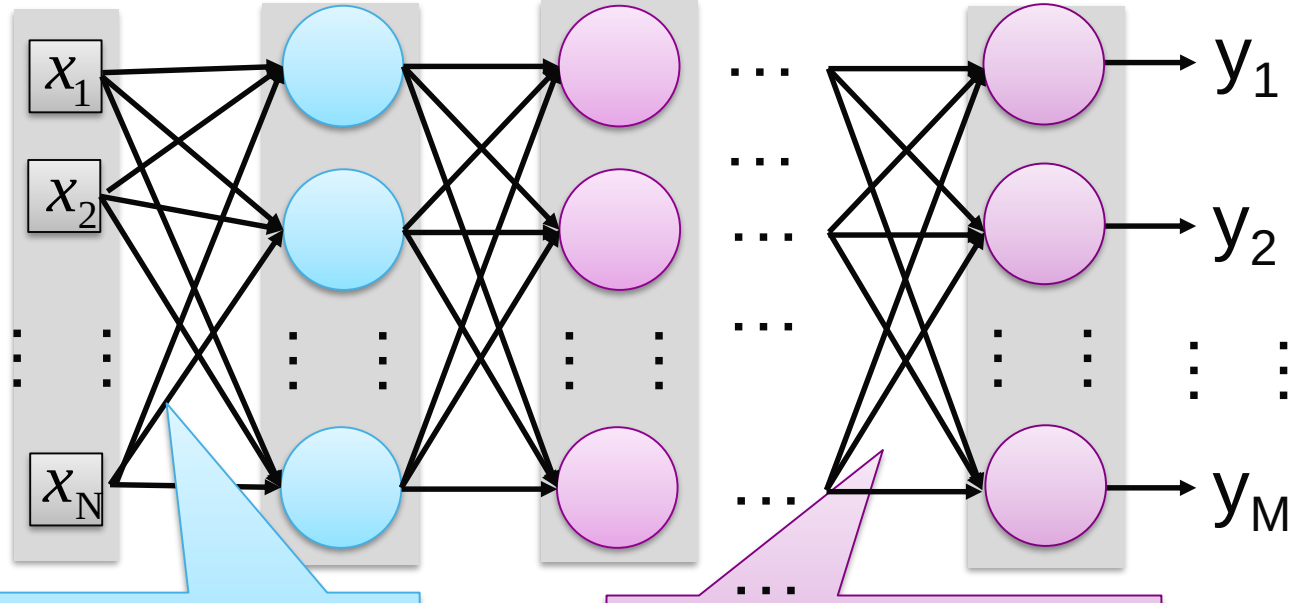
YES

NO

Good Results on Training Data?

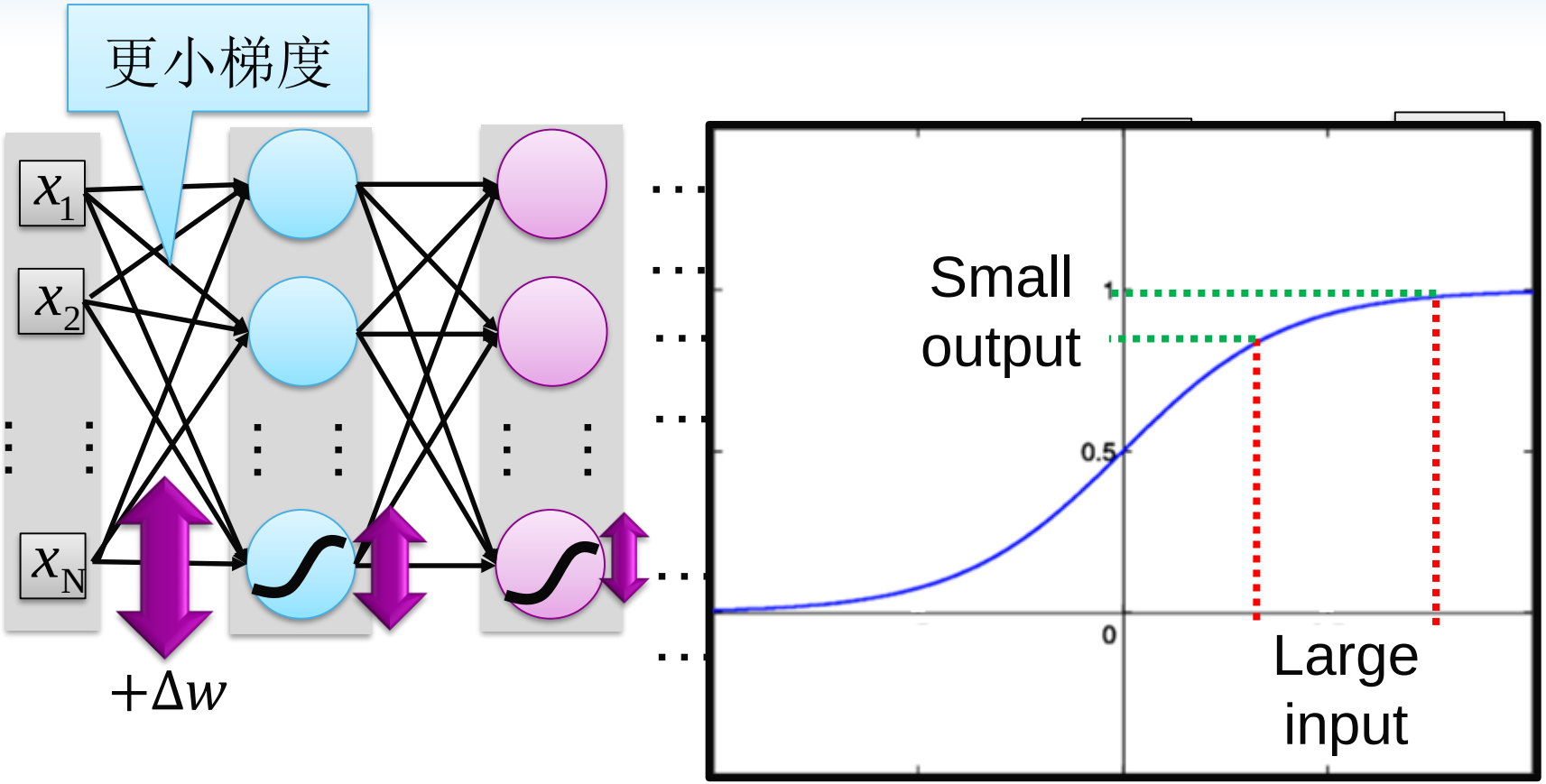
- 新的激活函数
- 合适的损失函数
- 自适应的学习率、动量

梯度消失问题



更小梯度	更大梯度
学得非常慢	学的非常快
几乎随机	已经收敛

梯度消失问题

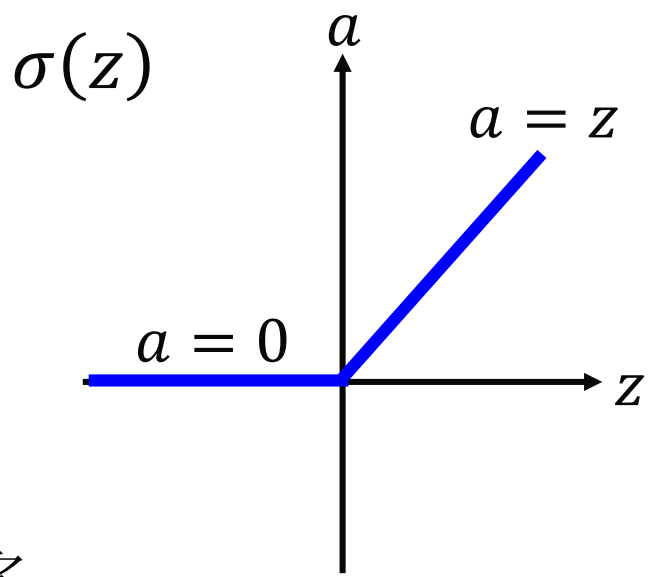


计算导数的直观方法 …

$$\frac{\partial l}{\partial w} =? \frac{\Delta l}{\Delta w}$$

新的激活函数

Rectified Linear Unit (ReLU)



原因：

1. 快速计算

2. 符合生物神经元特征

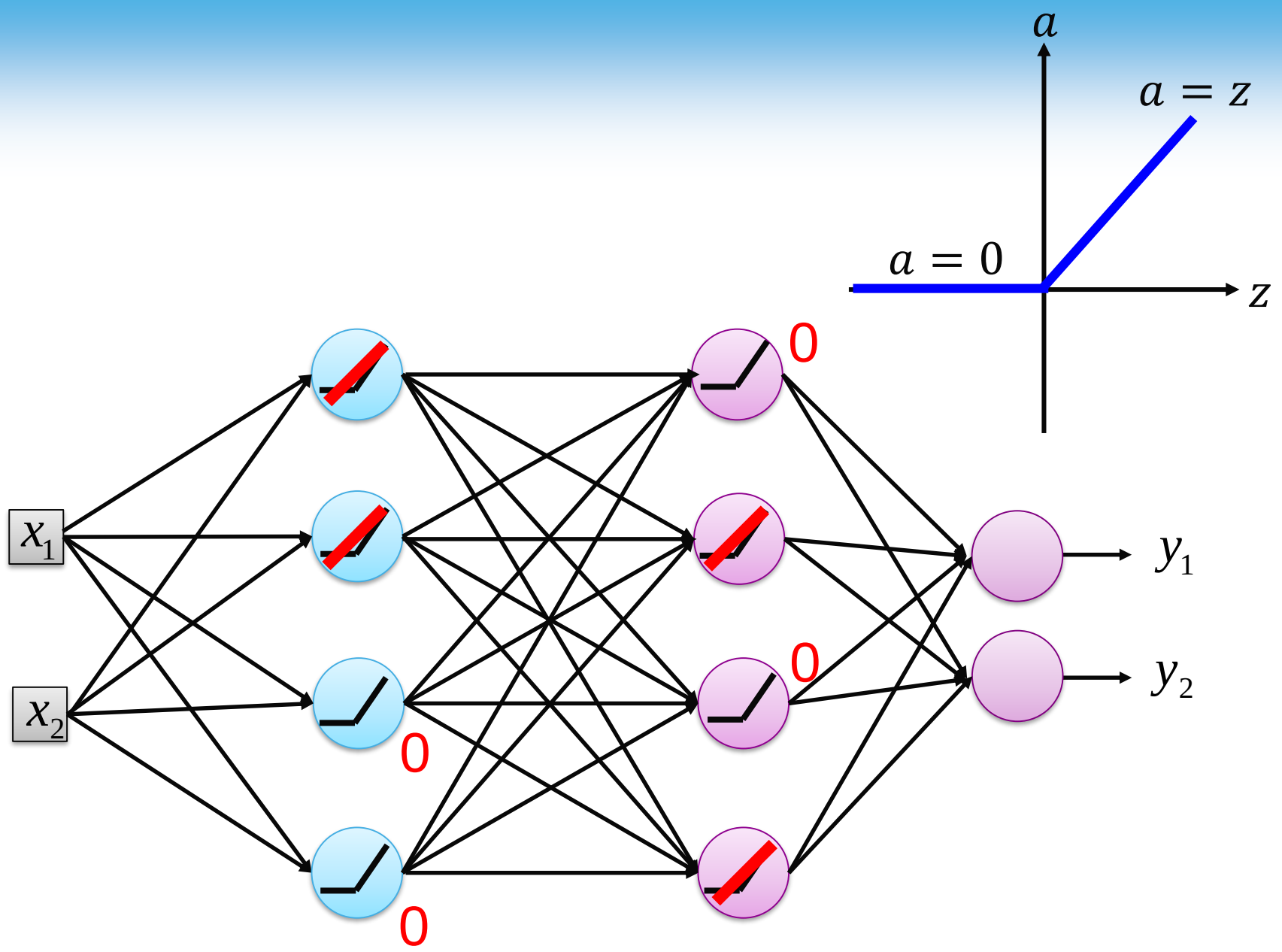
3. Infinite sigmoid with different biases

4. 解决梯度消失问题

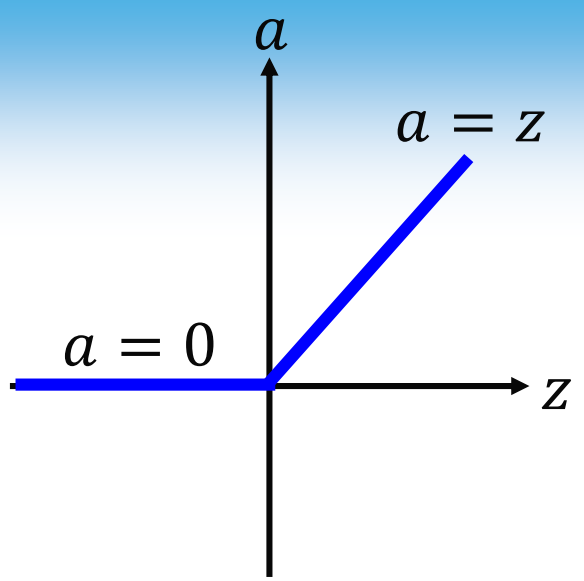
更多：

一. https://keras.io/activations/

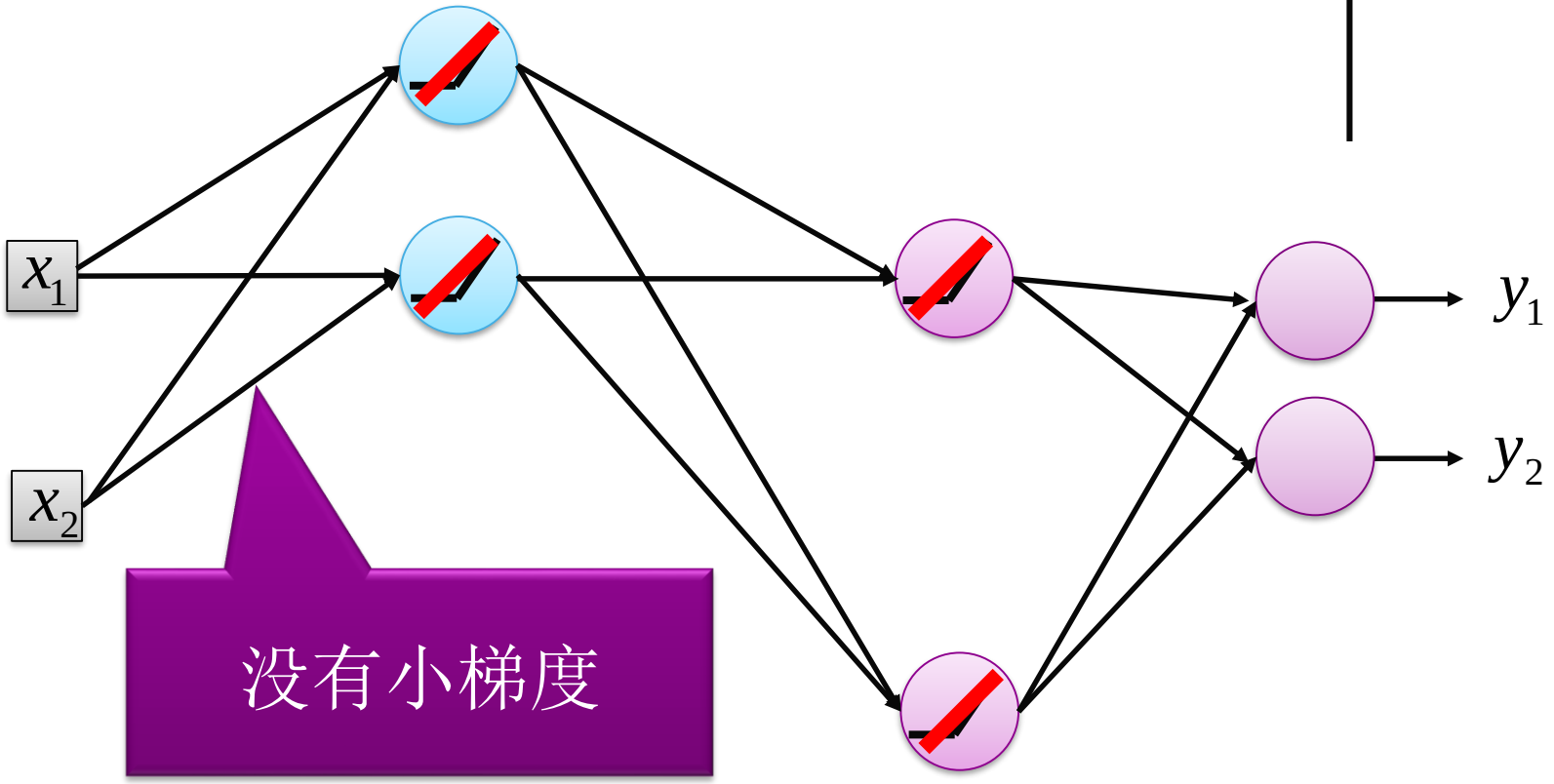
ReLU



ReLU

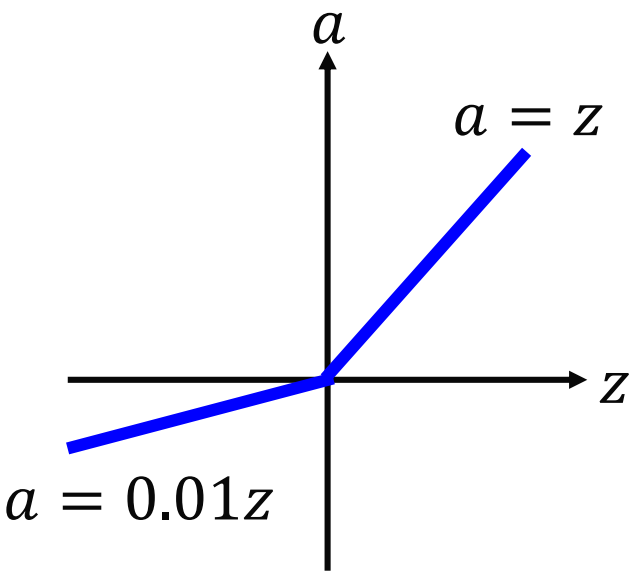


更瘦的线性网络

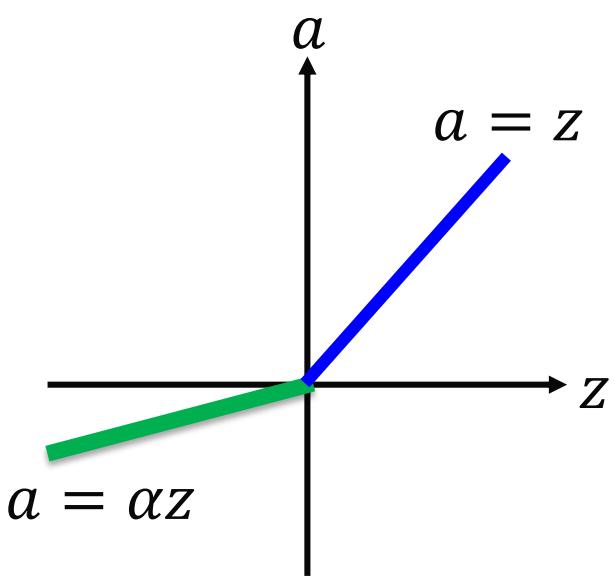


ReLU － 变体

Leaky ReLU

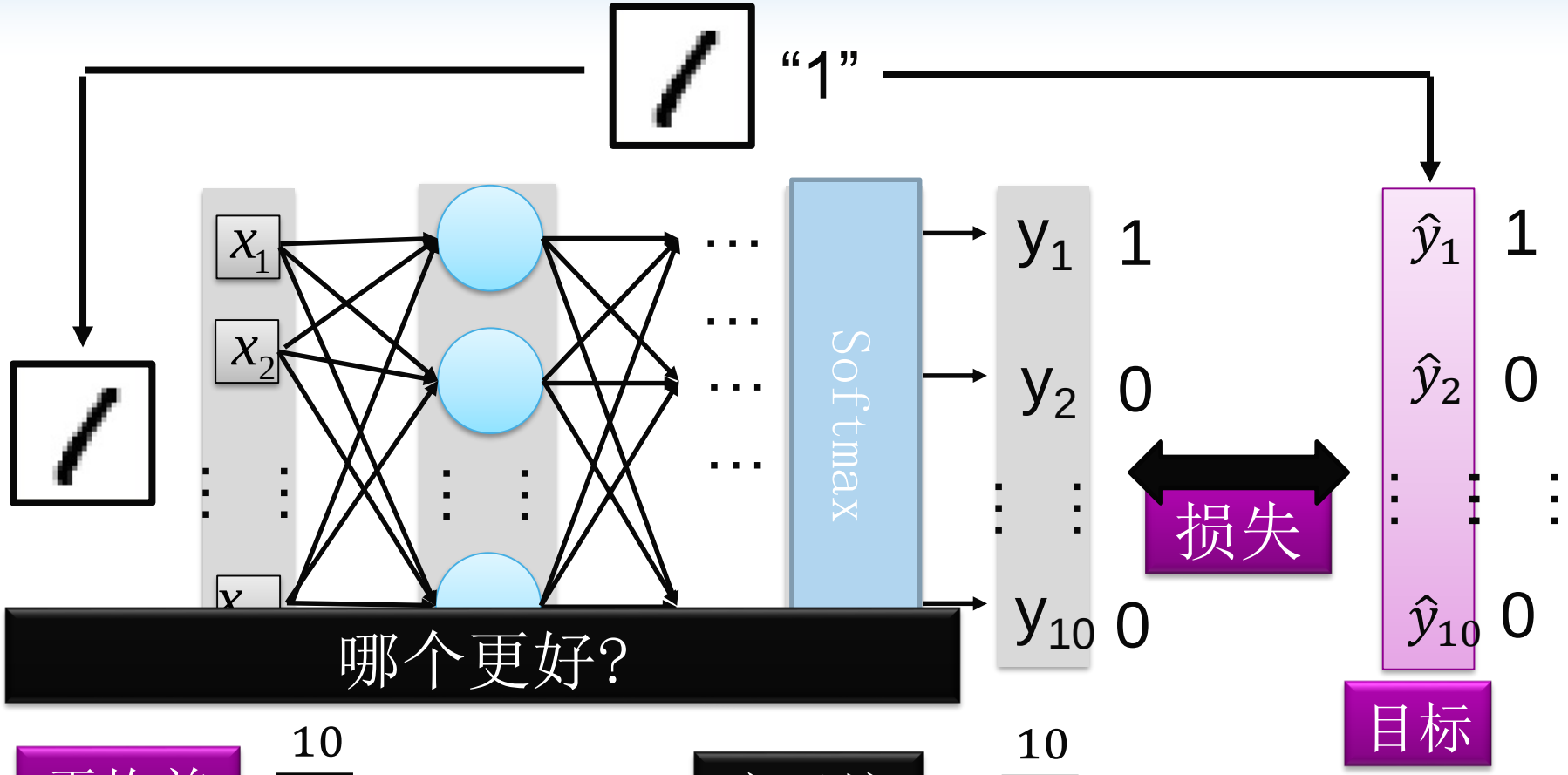


Parametric ReLU



α 也用梯度下降法学习

合适的损失函数

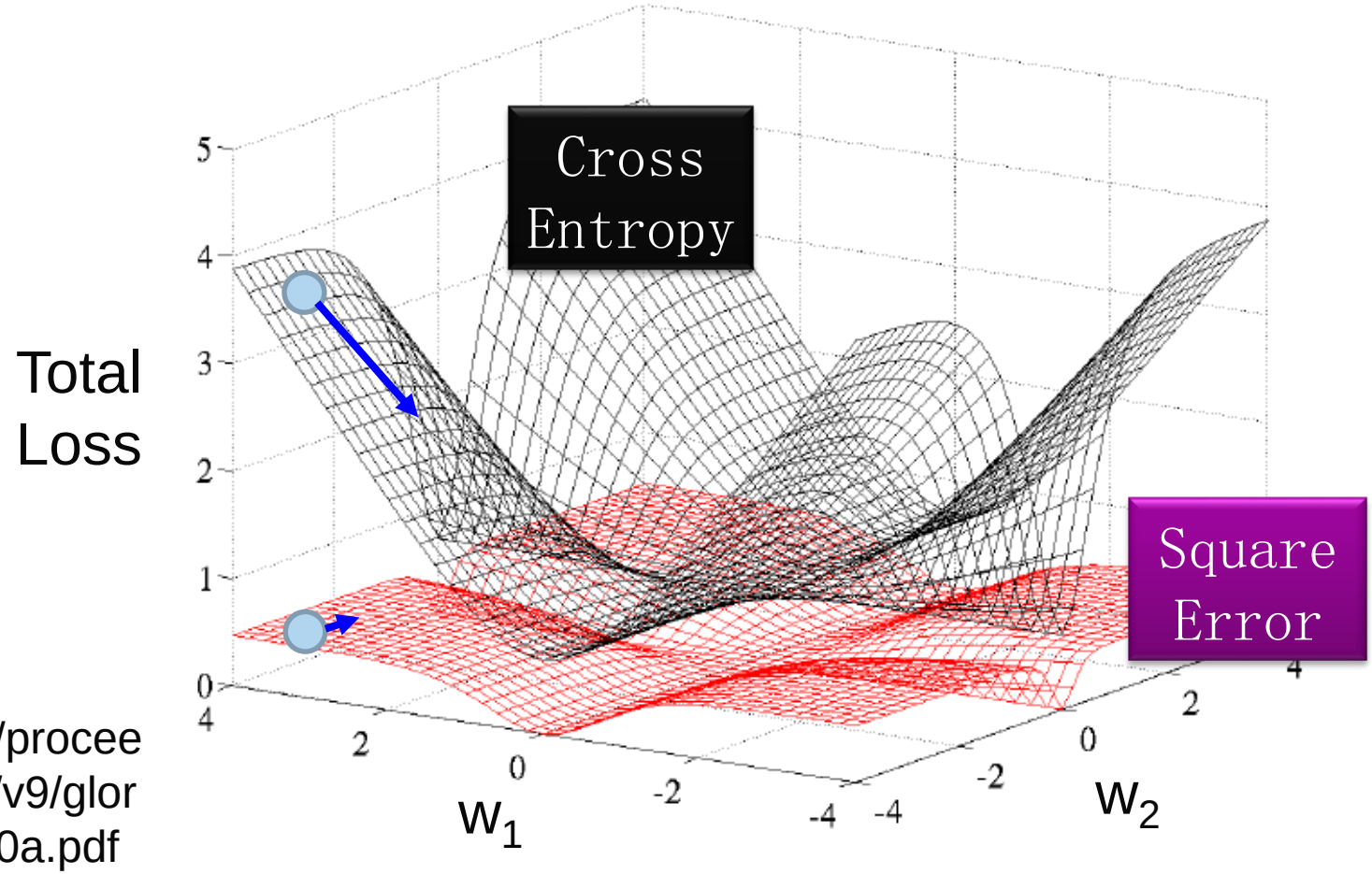


平均差 $\sum_{i=1}^{10}(y_i-\hat{y}_i)^2$ =0

交叉熵 $-\sum_{i=1}^{10}\hat{y}_i ln y_i$ =0

选择合适的损失函数

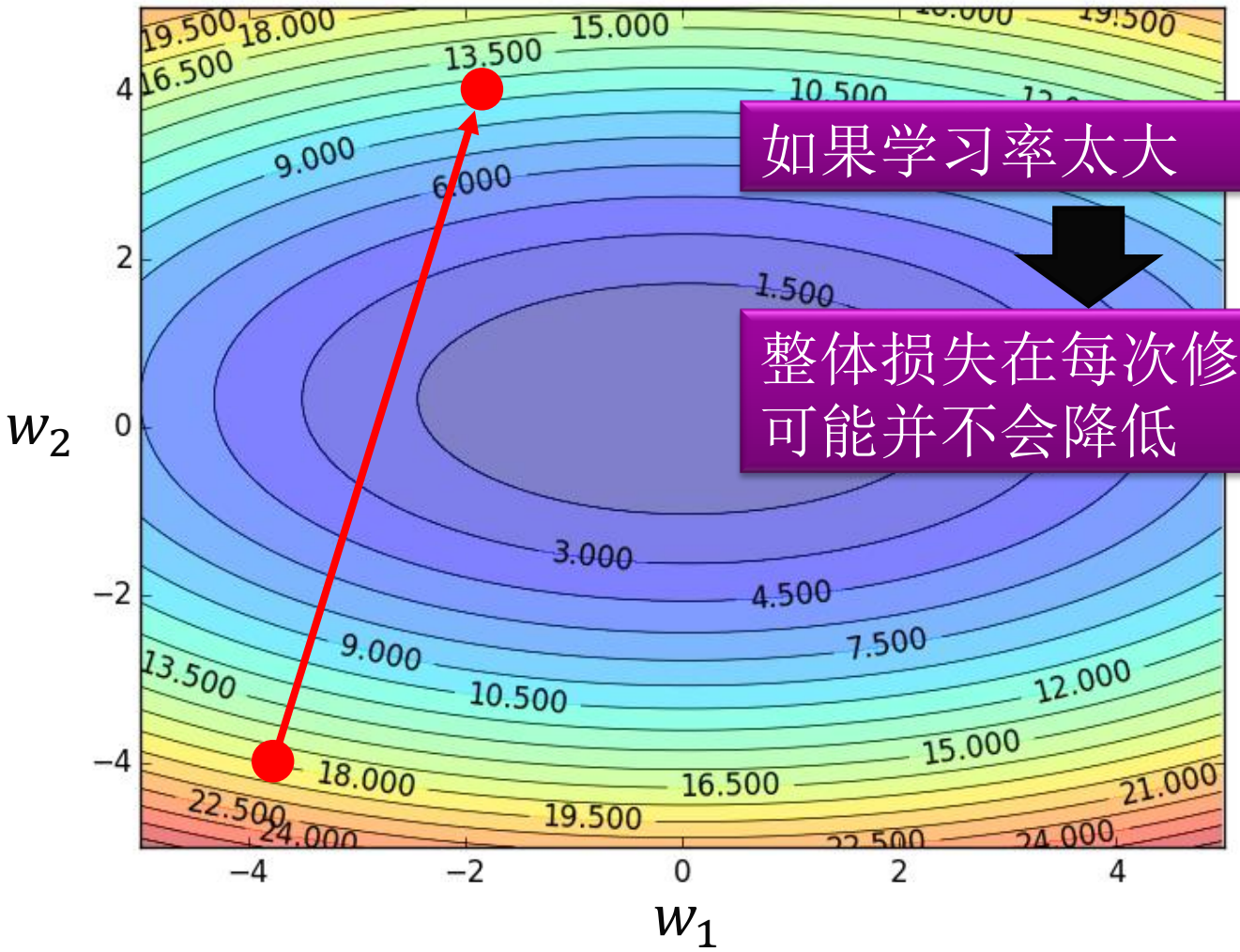
当使用softmax输出层时，使用交叉熵损失函数



http://jmlr.org/proceedings/papers/v9/glorot10a/glorot10a.pdf

学习率

小心设置学习率 η

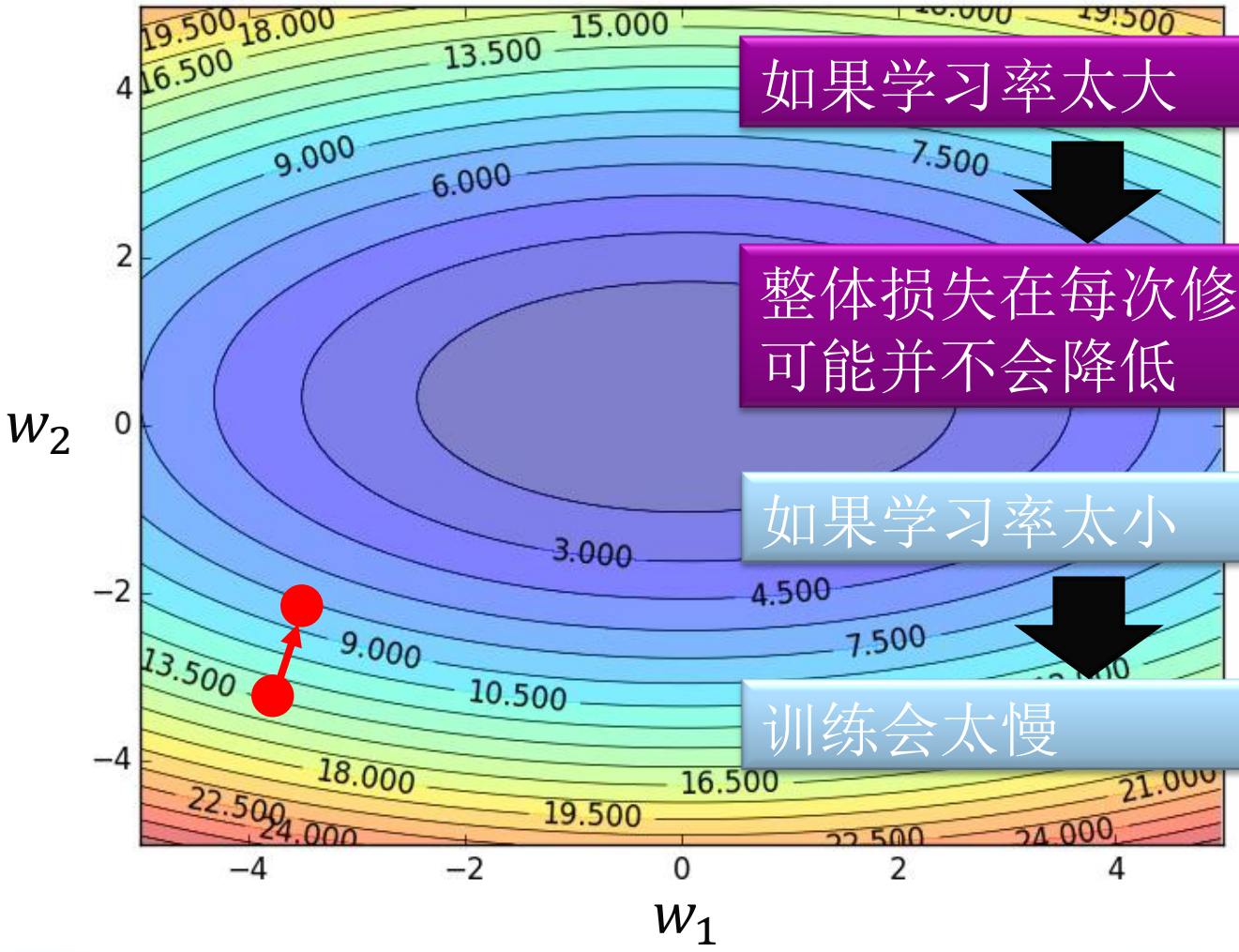


如果学习率太大

整体损失在每次修改之后可能并不会降低

学习率

小心设置学习率 η



如果学习率太大

整体损失在每次修改之后可能并不会降低

如果学习率太小

训练会太慢

学习率

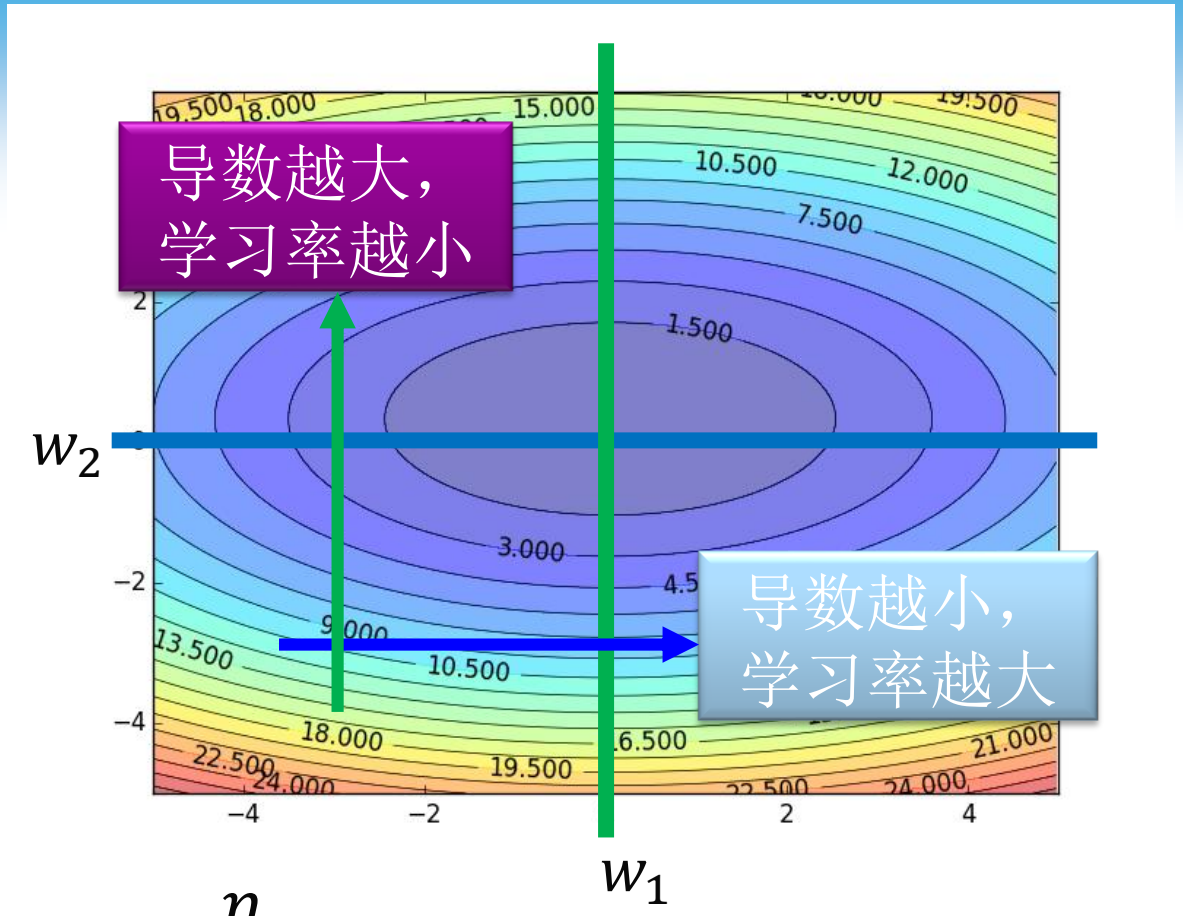
- 简单常用的想法：每几个epoch就将学习率减小一定幅度
- 一开始，我们离目标很远，可以使用较大的学习率
- 训练几个epoch后，我们接近目标了，就减小学习率

  如：$\eta^t = \eta/\sqrt{t+1}$

学习率不可能是one-size-fits-all

- 不同的参数应该有不同的学习率

自适应的学习率

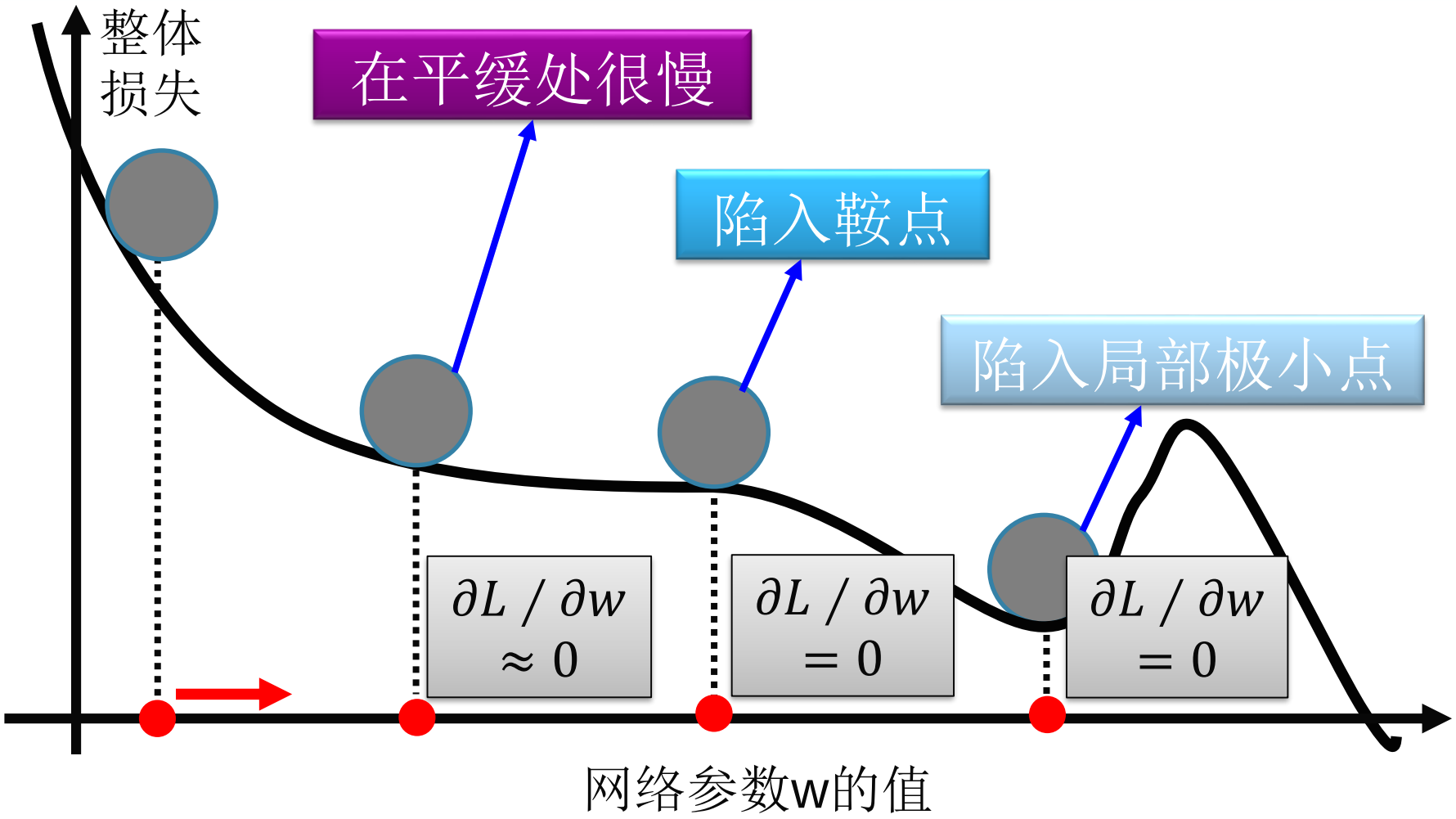


Adagrad

$$w^{t+1} \leftarrow w^t - \frac{\eta}{\sqrt{\sum_{i=0}^{t}(g^i)^2}} g^t$$

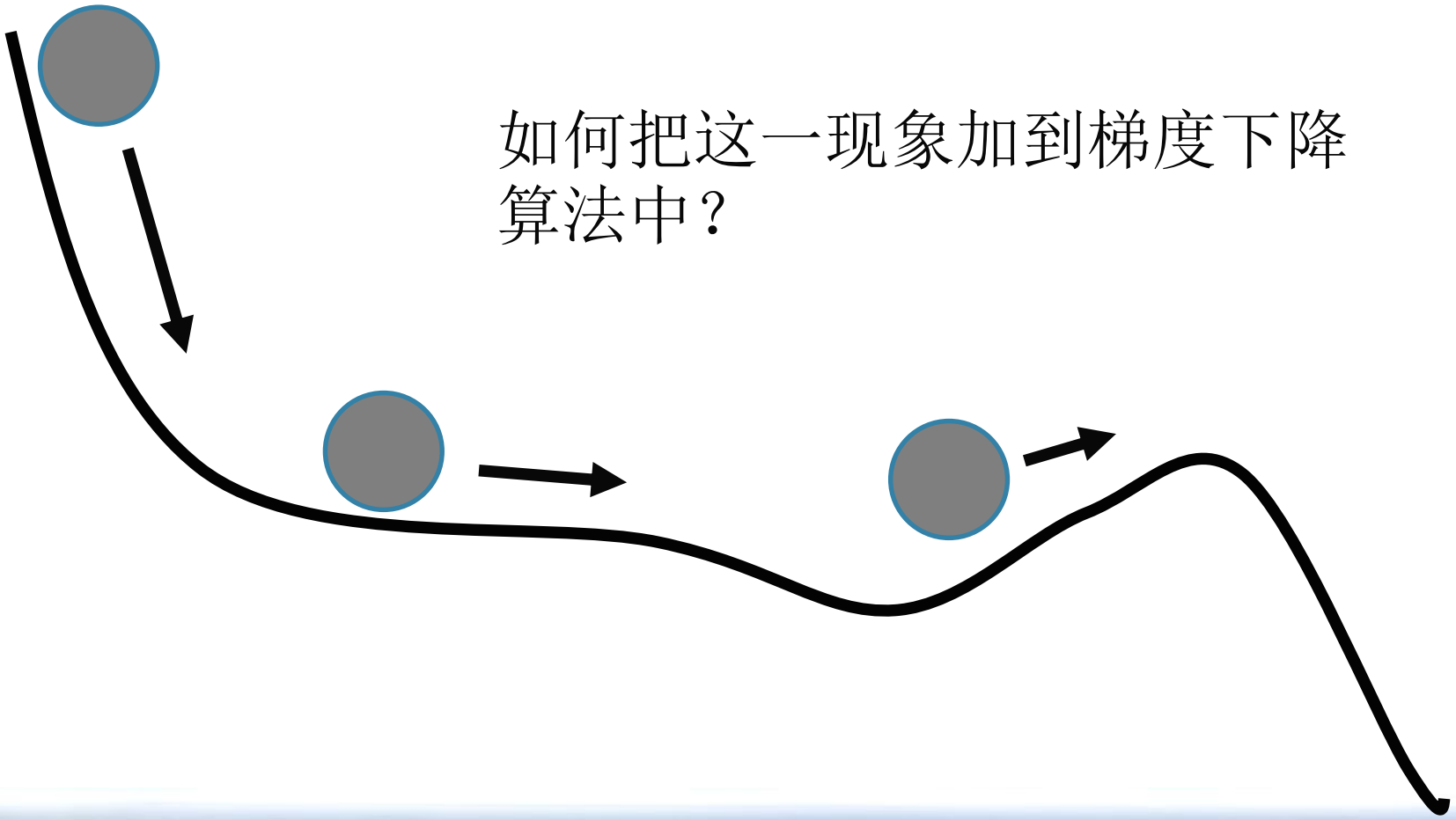
Adam: *RMSProp*（改进的Adagrad）+ 动量

难以发现最优的网络参数



在物理世界中……

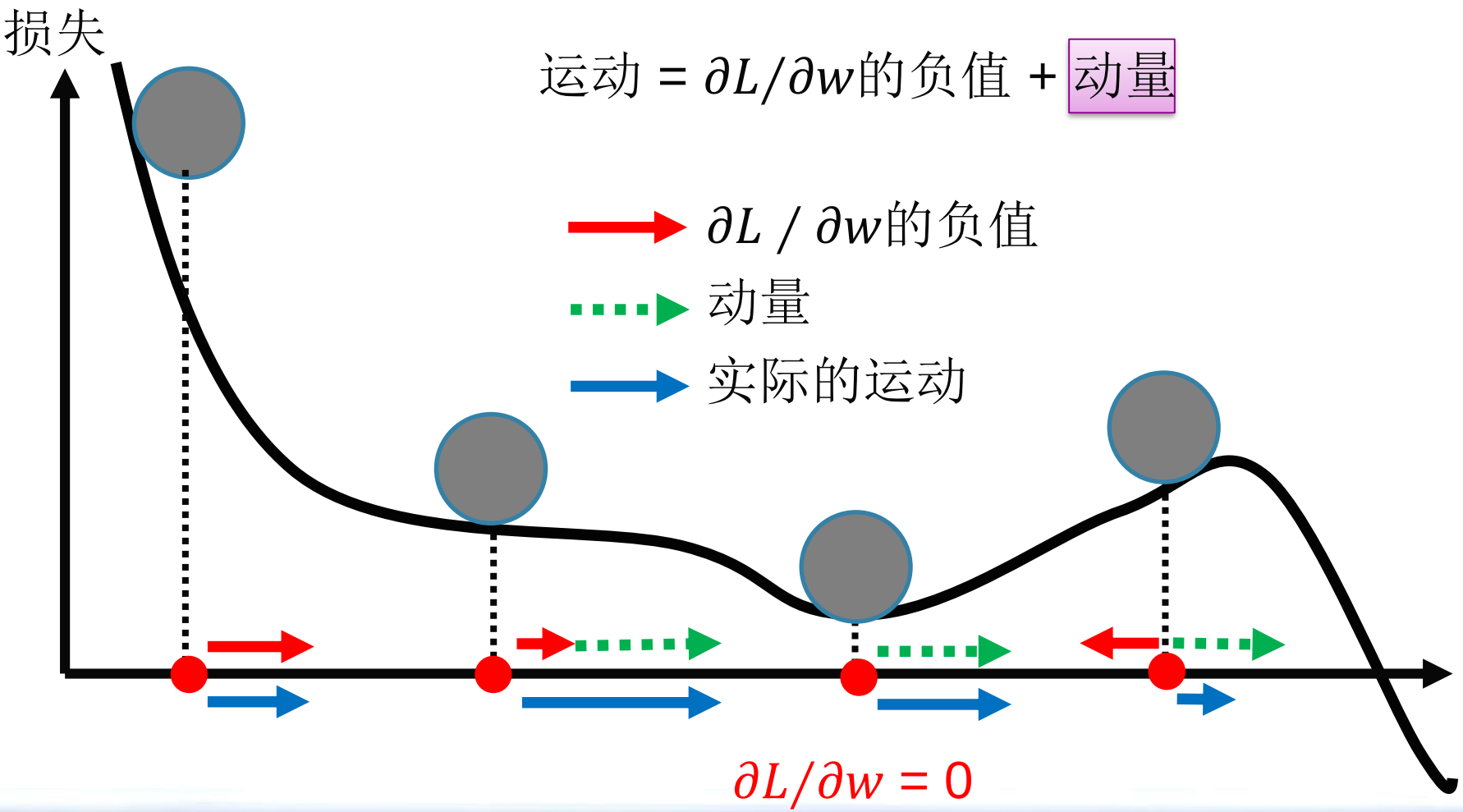
动量



如何把这一现象加到梯度下降算法中？

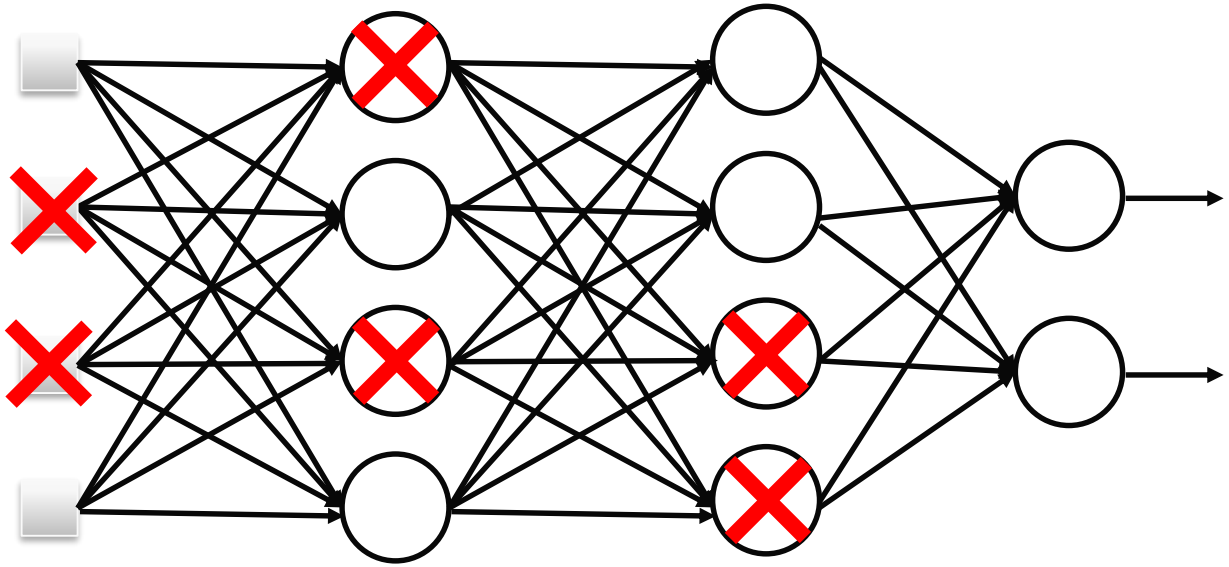
动量（Momentum）

仍然不能保证到达全局最优，
但增加了一些希望 ……



Dropout

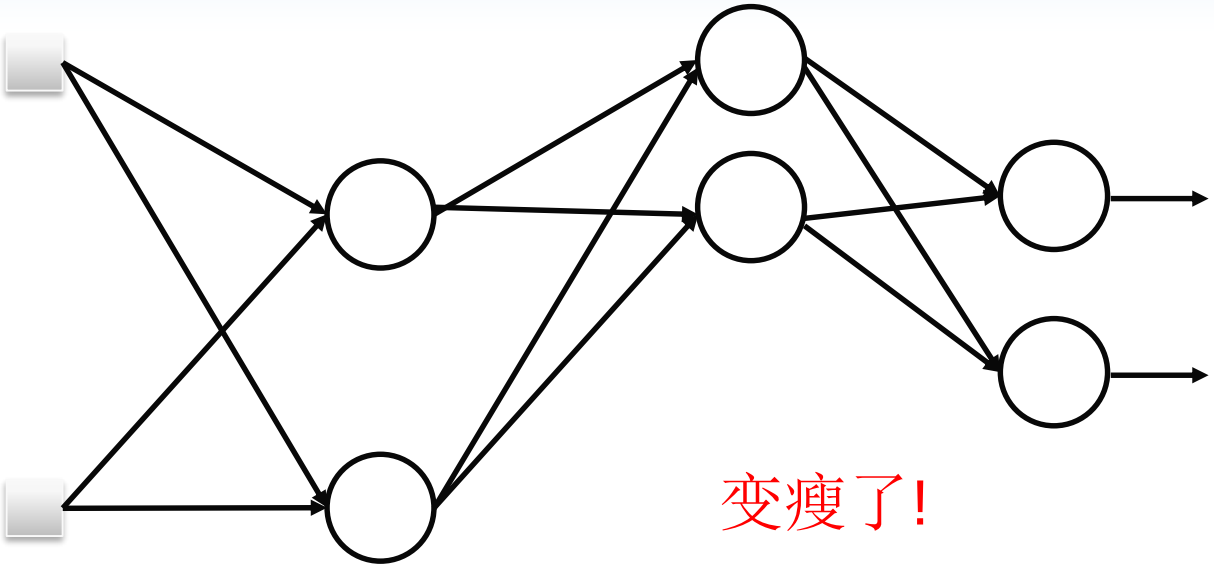
训练：



- **每次修改参数值前**
  - 每个神经元有p%的概率退出

Dropout

训练：



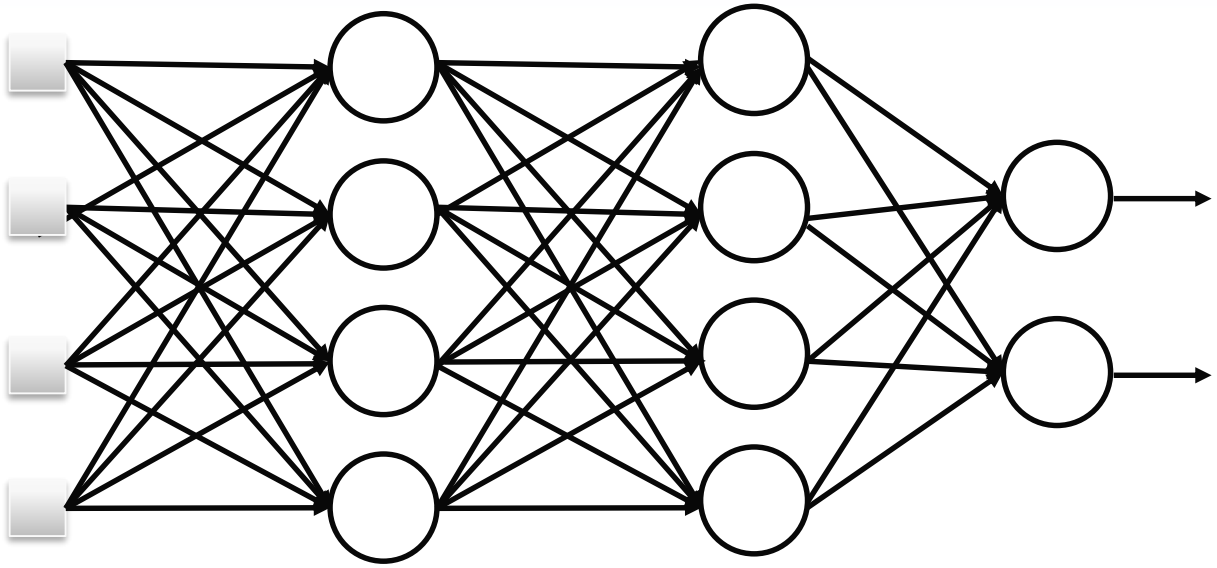
- **每次修改参数值前**
  - 每个神经元有p%的概率退出

    ⇒ **网络的结构变了**
  - 训练新的网络

对每个mini-batch，我们重新随机选取退出的神经元

Dropout

测试：



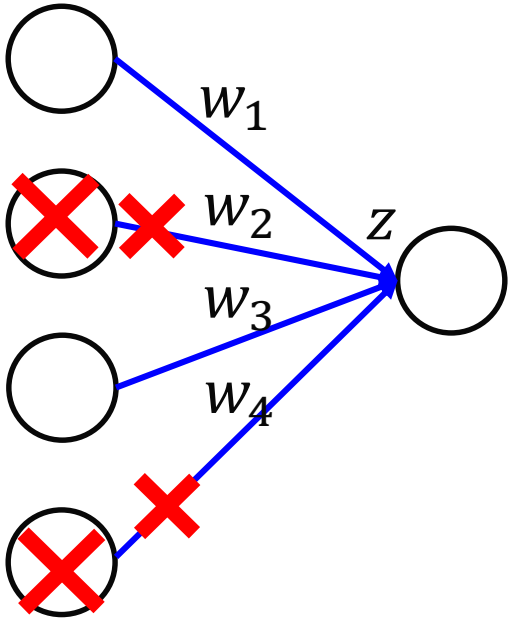
- **没有dropout**
  - 如果训练时的退出概率是p%, 则所有的权重都乘以（1-p%）
  - 假设退出概率是50%，
    如果一个权重w训练得到的值是1，则测试时将*w*的值设为0.5

Dropout - 直观理解

为什么测试时网络的权重应该乘以(1-p)% (dropout rate)?

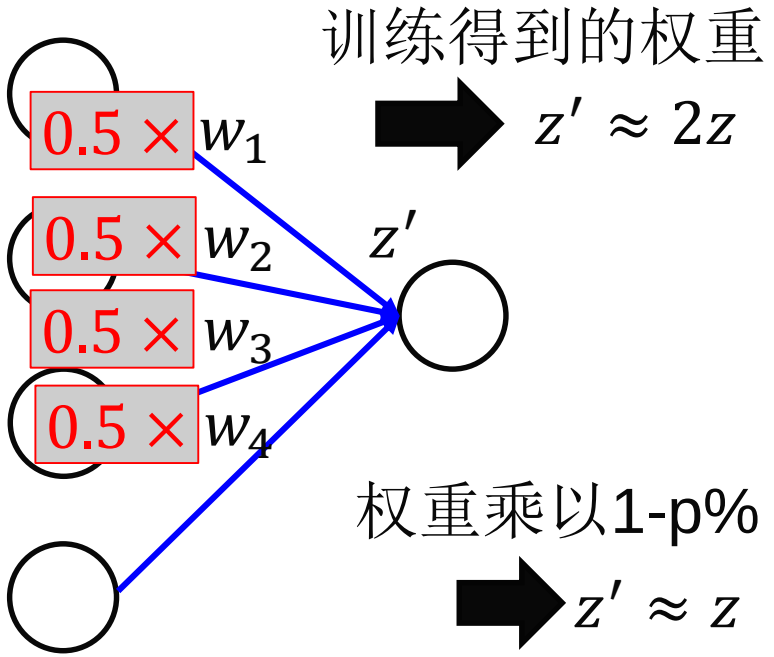
Training of Dropout

假设退出率是50%



Testing of Dropout

没有退出



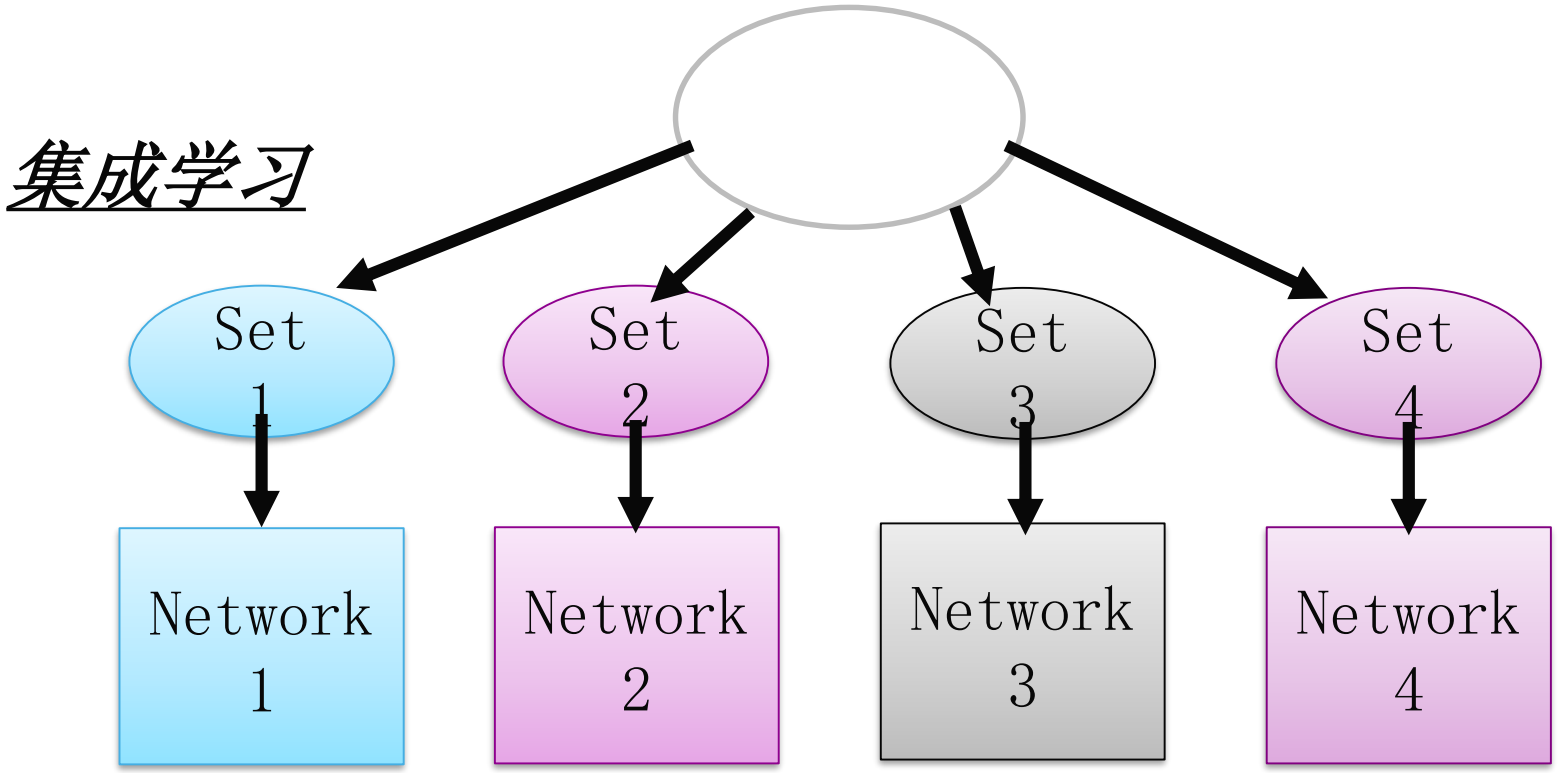
训练得到的权重

$z' \approx 2z$

权重乘以1-p%

$z' \approx z$

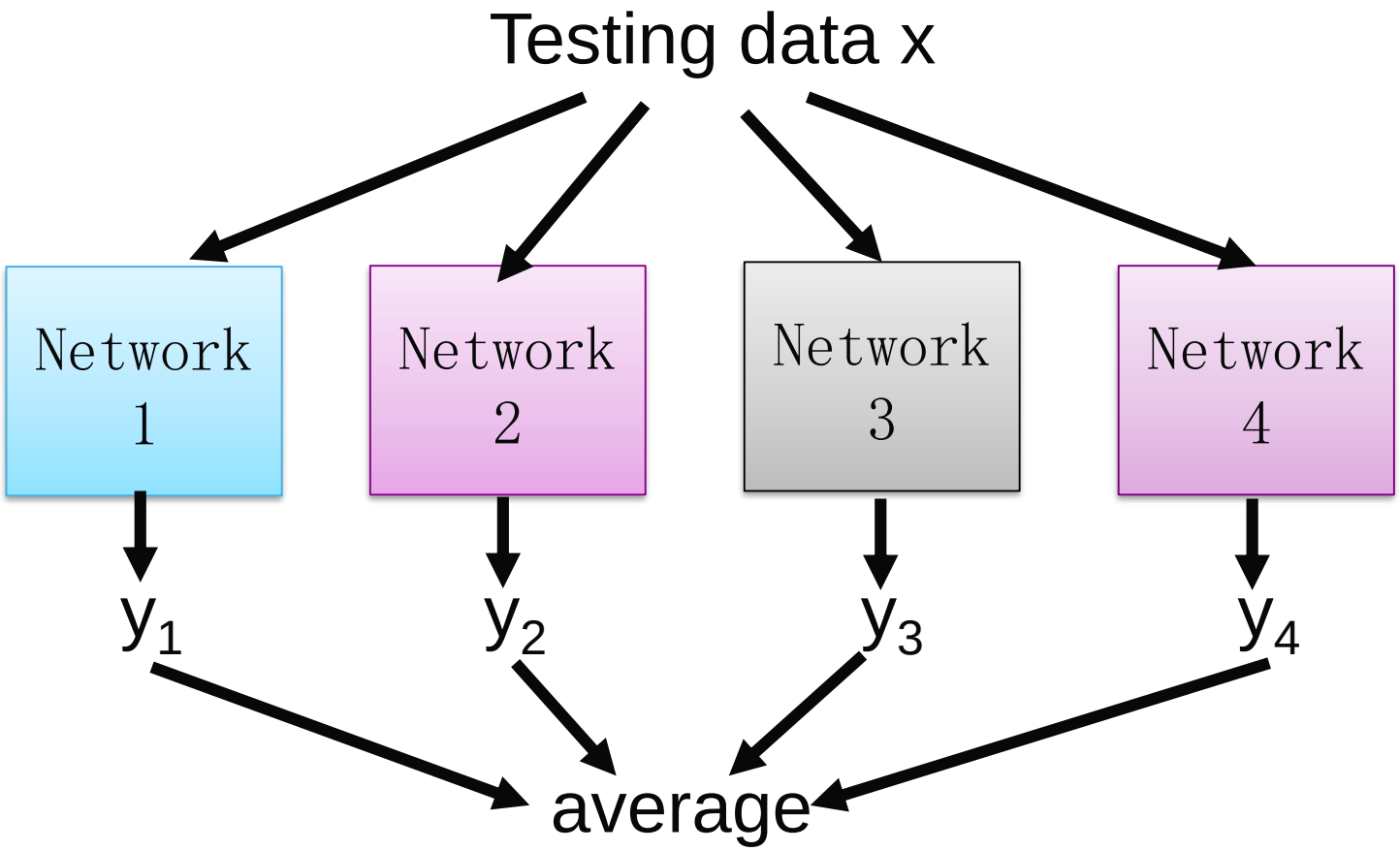
Dropout是一种集成学习



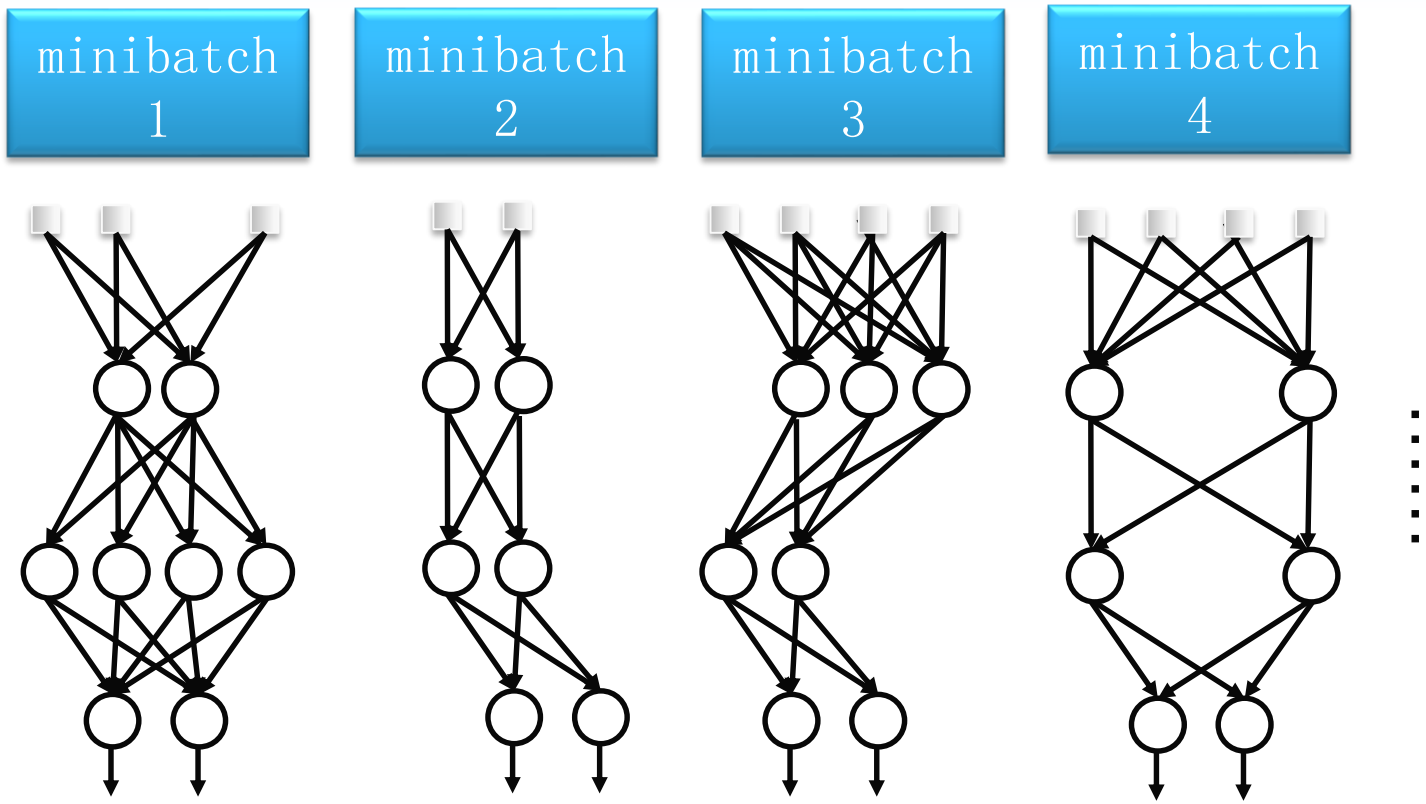
训练一组不同结构的网络

Dropout是一种集成学习

集成学习



Dropout是一种集成学习



训练过程

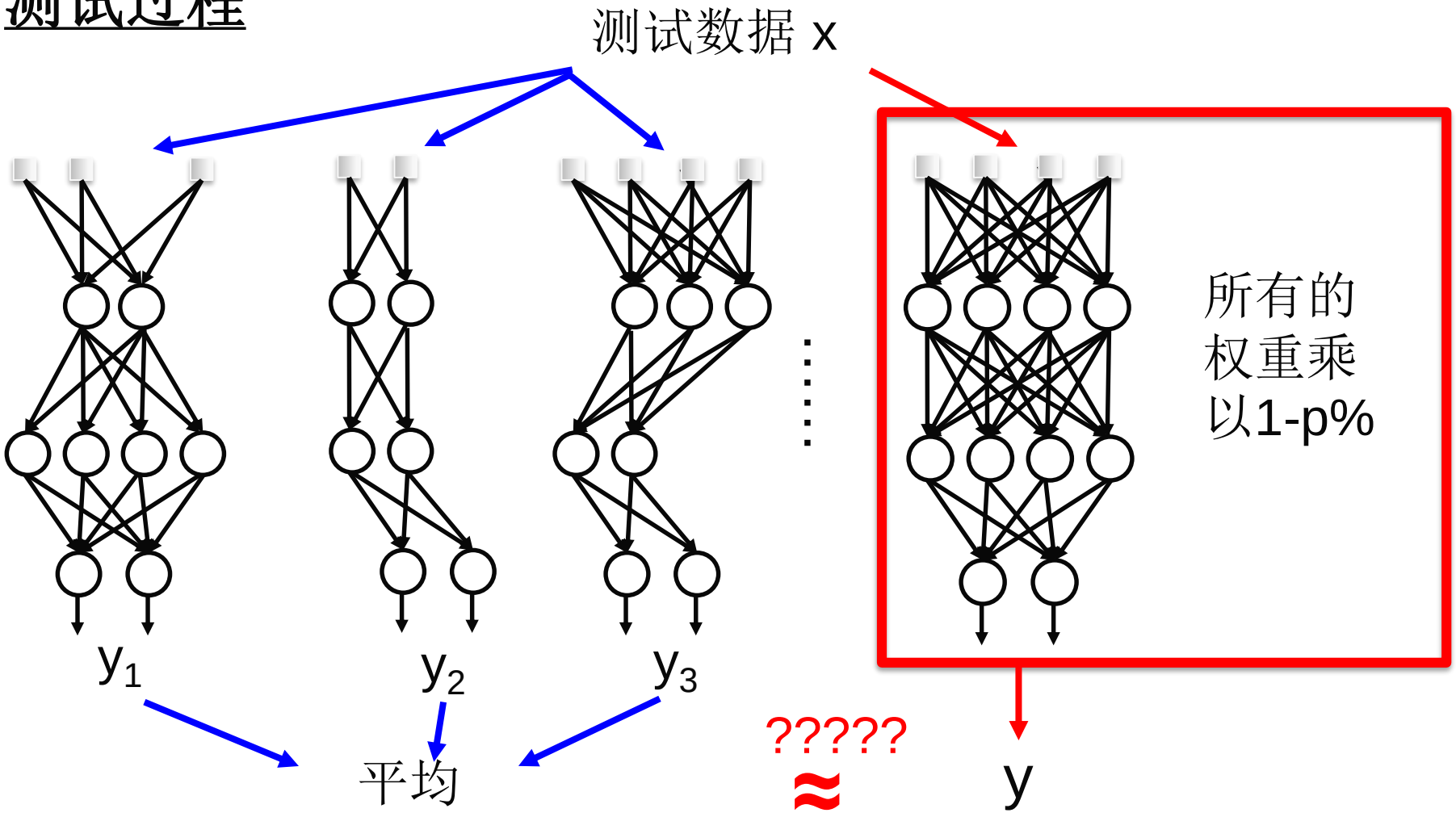
M 神经元

2^M 可能的网络

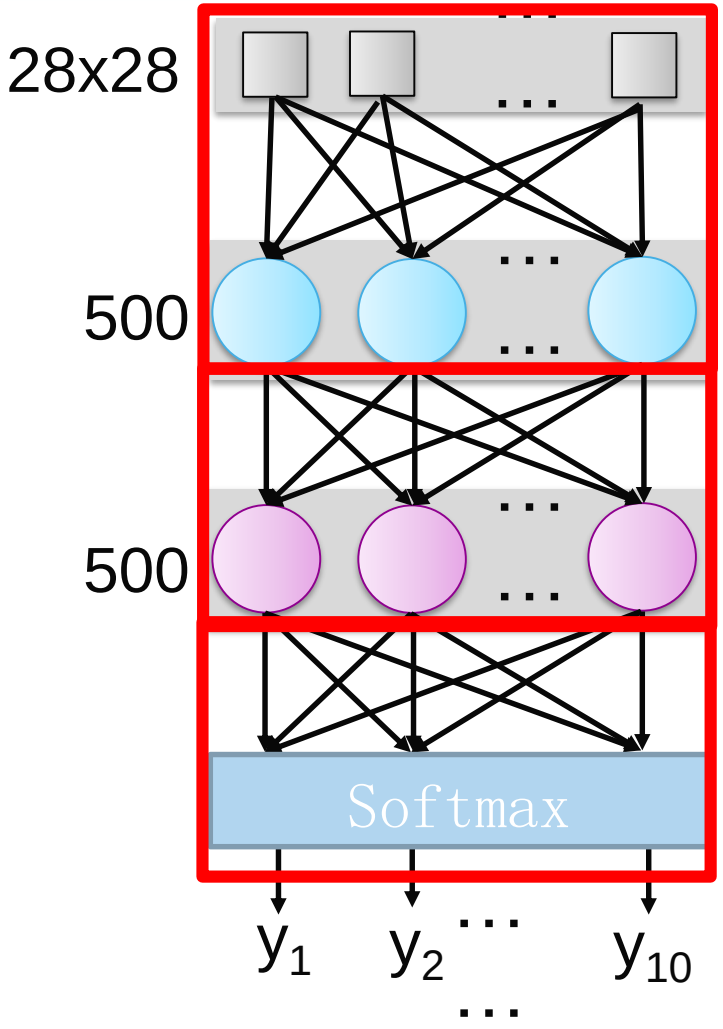
- 用一个mini-batch训练一个网络
- 网络中的一些参数是共享的

Dropout是一种集成学习

测试过程



示例




```
from keras.models import Sequential
from keras.layers import Dense,Activation

model = Sequential()
```

```
model.add(Dense(500, input_dim=784))
model.add(Activation('sigmoid'))
```

```
model.add( dropout(0.8) )
```

```
model.add(Dense(500))
model.add(Activation('sigmoid'))
```

```
model.add( dropout(0.8) )
```

```
model.add(Dense(10))
model.add(Activation('softmax'))
```

```
model.summary()
```

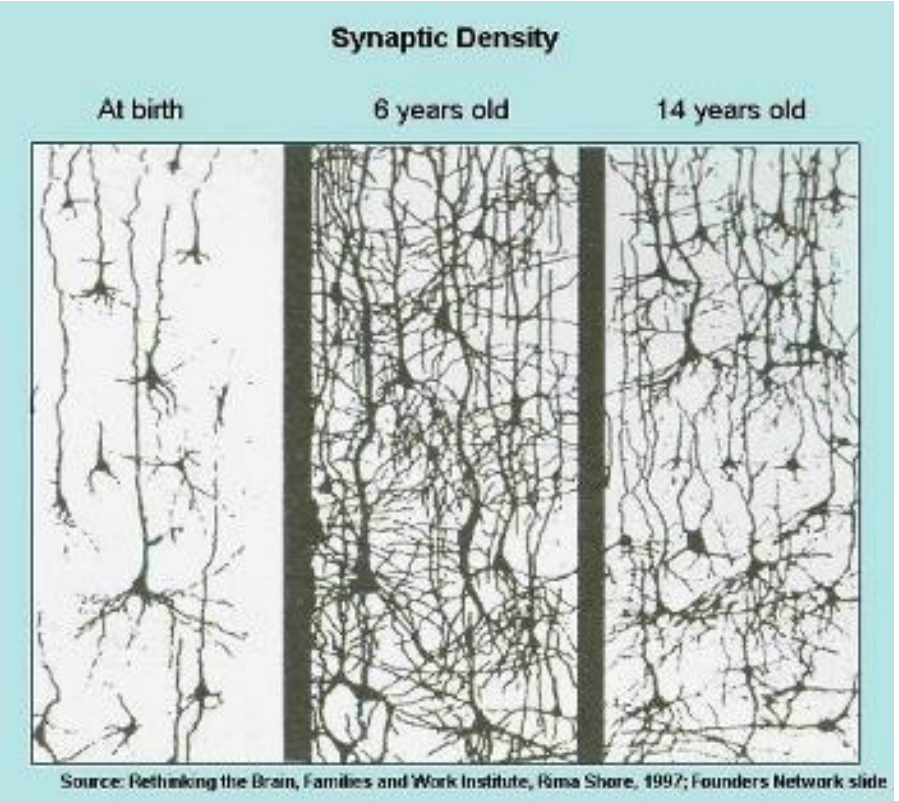
正则化 – 权重衰减

正则化即在损失函数中加入一个正则项。

直观地，正则化的作用是让网络偏好学习更小的权值，即复杂度低的网络。

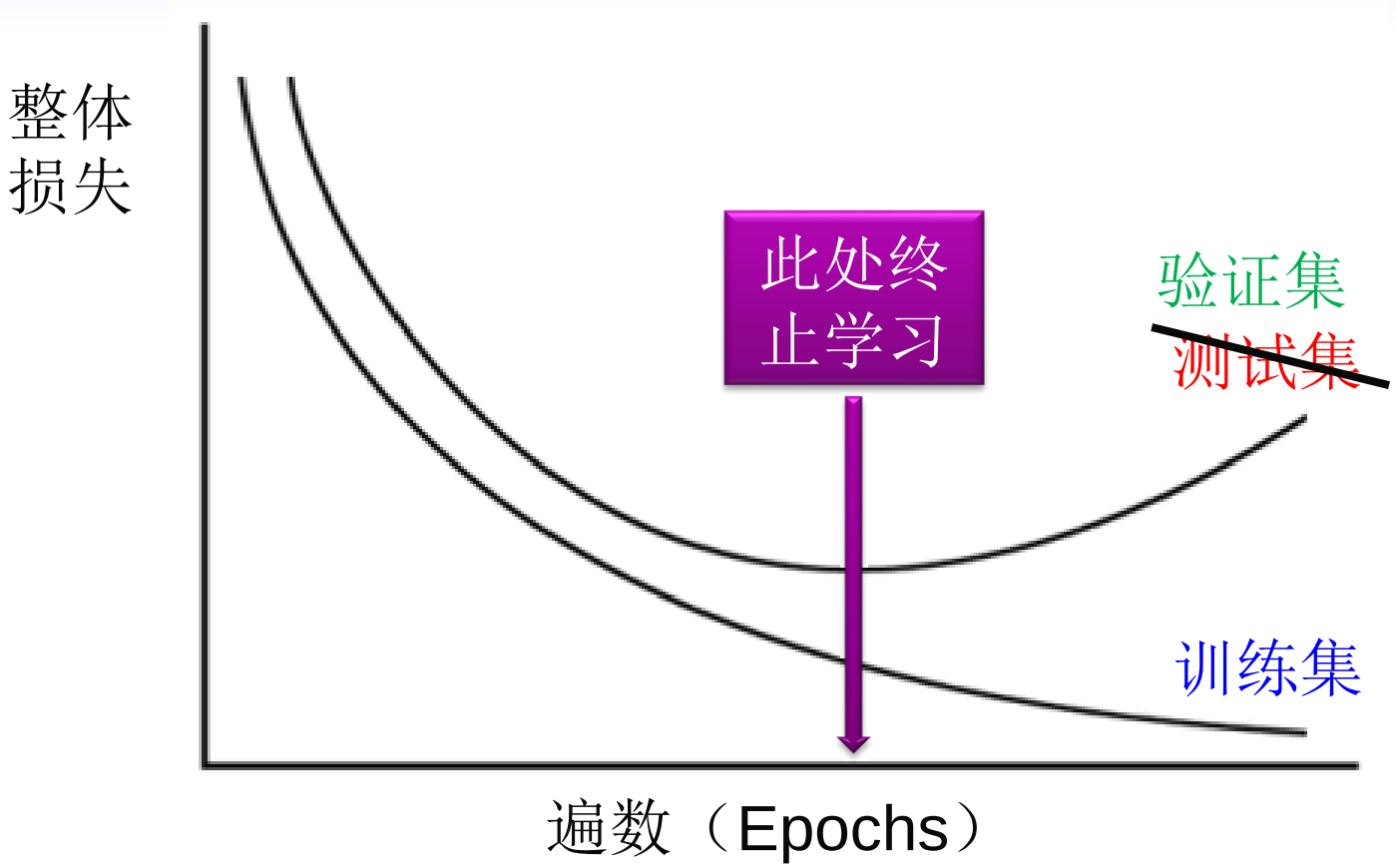
https://keras.io/regularizers/

人脑会将神经元之间
没用的连接滤去



人工神经网络类似地可以通过权重衰减改善性能

提前终止



Keras: http://keras.io/getting-started/faq/#how-can-i-interrupt-training-when-the-validation-loss-isnt-decreasing-anymore